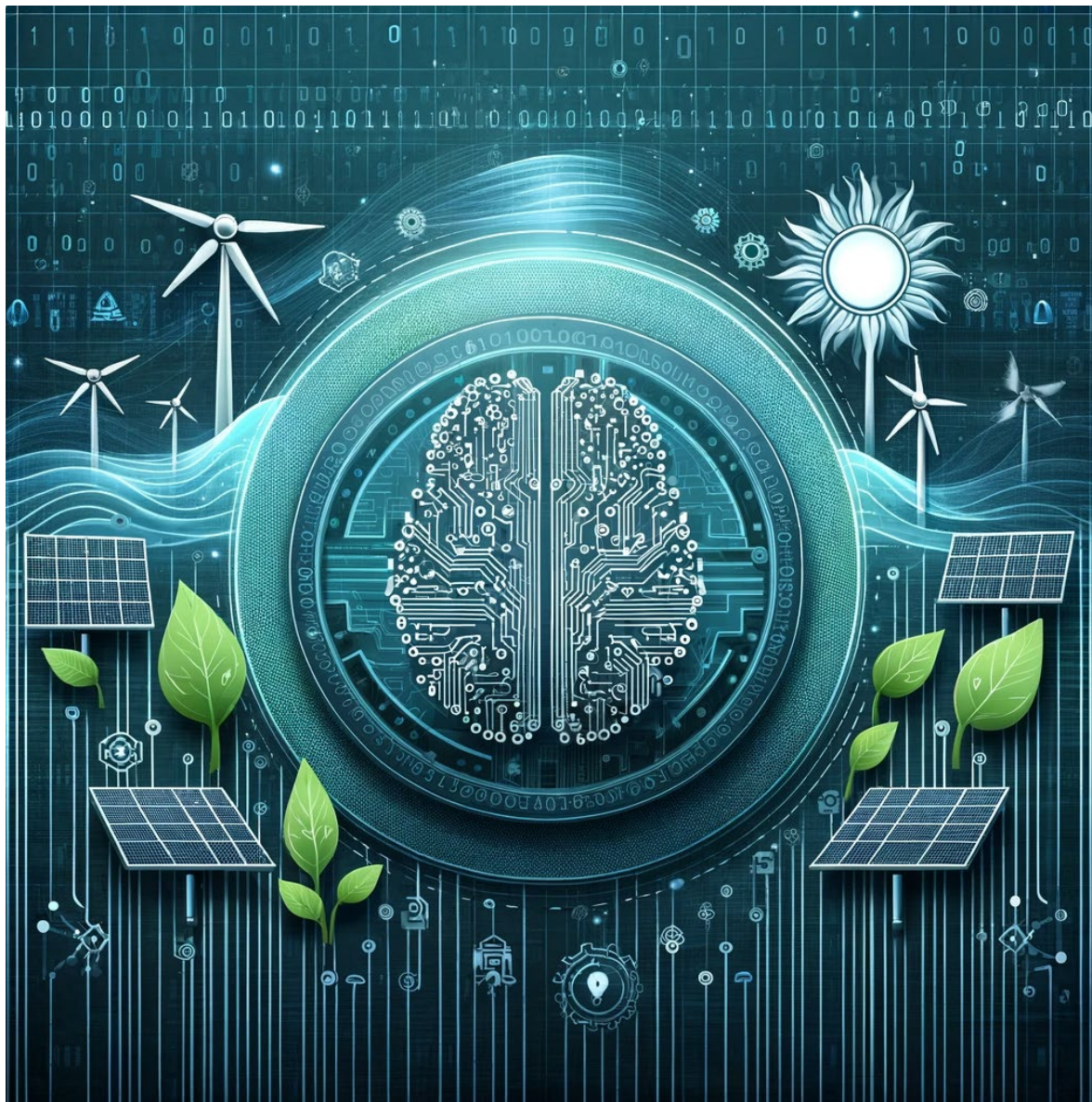


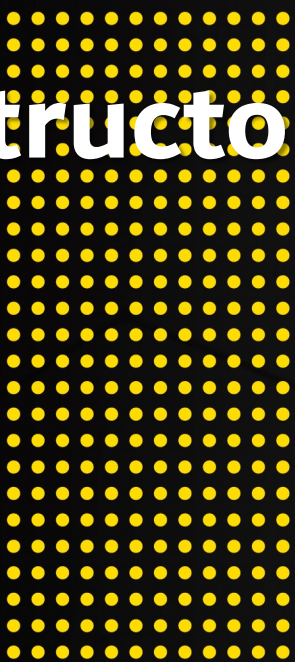


# Analítica de Datos con IA, énfasis en energías renovables

INGEOEXPERT

2024

# Instructor

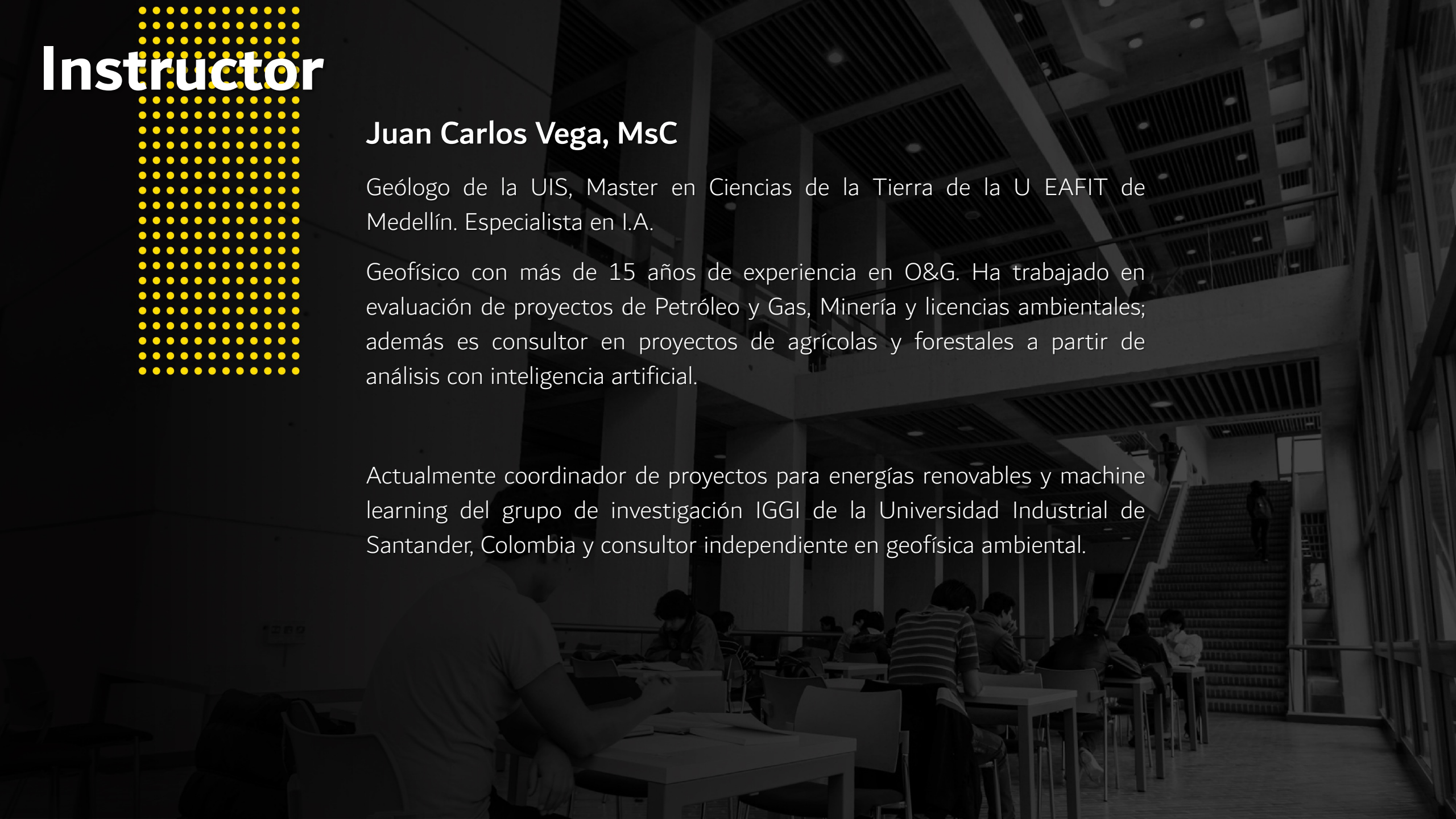


## Juan Carlos Vega, MsC

Geólogo de la UIS, Master en Ciencias de la Tierra de la U EAFIT de Medellín. Especialista en I.A.

Geofísico con más de 15 años de experiencia en O&G. Ha trabajado en evaluación de proyectos de Petróleo y Gas, Minería y licencias ambientales; además es consultor en proyectos de agrícolas y forestales a partir de análisis con inteligencia artificial.

Actualmente coordinador de proyectos para energías renovables y machine learning del grupo de investigación IGGI de la Universidad Industrial de Santander, Colombia y consultor independiente en geofísica ambiental.



# OPTIMIZACION

## Conceptos y aplicaciones



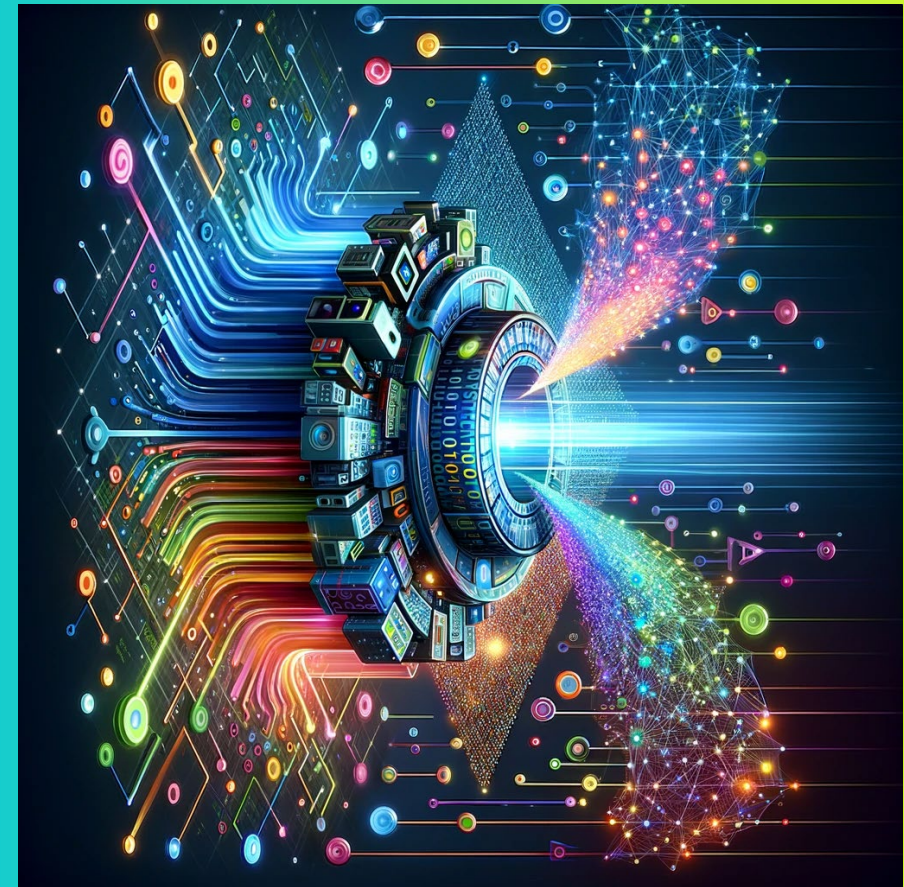
# **Que es la optimización en ML**

Definición y conceptos básicos

# Optimización en Machine learning

## Conceptos

La optimización del aprendizaje automático es el proceso de ajustar los parámetros y la estructura de un modelo de aprendizaje automático para mejorar su rendimiento en una tarea específica. Esto implica seleccionar los mejores algoritmos, ajustar los hiperparámetros y elegir representaciones de características apropiadas para minimizar el error del modelo o maximizar su precisión, evitando al mismo tiempo el sobreajuste y manteniendo la generalización a datos invisibles.



# ¿Porque es importante la optimización en ML?

Conceptos



- Rendimiento mejorado
- Uso eficiente de recursos
- Generalización del modelo
- Gestión de compensaciones
- Personalización

# Técnicas esenciales de optimización en Machine learning

## Conceptos

- Exploración por grilla
- Exploración Aleatoria
- Exploración Bayesiana
- Enjambre de partículas
- Recocido simulado o “Annealing”
- Algoritmos genéticos
- Entrenamiento basado en la población.



# Optimización por grillas:

## Conceptos

La búsqueda por Grilla es una técnica frecuente para la optimización de hiperparámetros, que implica encontrar el mejor conjunto de hiperparámetros examinando todas las combinaciones posibles.

Este enfoque es más eficaz cuando ya se conoce el rango óptimo de hiperparámetros cruciales, ya sea a través de investigaciones empíricas, trabajos previos o estudios publicados. La desventaja es que este es el método más ineficiente y exigente desde el punto de vista computacional.

# Optimización por grillas:

## Conceptos

Por ejemplo, en un clasificador de máquina de vectores de soporte (SVM), si ha determinado seis hiperparámetros críticos (como núcleo, parámetro de regularización y grado) y tres valores potenciales para cada hiperparámetro dentro de un rango específico, la búsqueda en cuadrícula evaluará  $6 * 3 = 18$  modelos distintos para cada combinación única de hiperparámetros. Esto garantiza que nuestro conocimiento previo sobre el rango de hiperparámetros se integre en un conjunto limitado de evaluaciones de modelos.

# Optimización por grillas:

## ¿Como funciona?



### Definición de la Cuadrícula:

Se especifica una "cuadrícula" de hiperparámetros, lo que significa que se establecen diferentes valores para cada hiperparámetro que queremos probar.



**Evaluación de Modelos:** Para cada combinación única de hiperparámetros en esta cuadrícula, se entrena un modelo utilizando los hiperparámetros específicos y luego se evalúa su rendimiento mediante una métrica predefinida, como puede ser la precisión para tareas de clasificación o el error cuadrático medio para tareas de regresión.



### Selección del Mejor Modelo:

Después de evaluar todos los modelos posibles, se selecciona el conjunto de hiperparámetros que produjo el modelo con el mejor rendimiento.



# Optimización por grillas:

## Aplicación optima



El espacio de búsqueda de hiperparámetros no es demasiado grande.



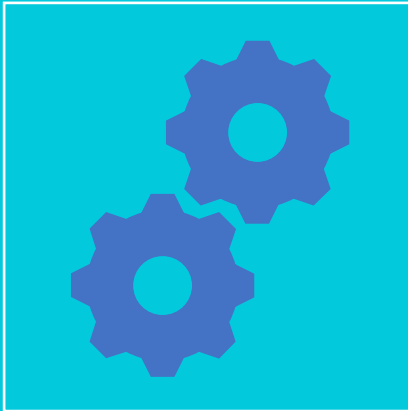
Se tiene un buen conocimiento previo de los rangos de hiperparámetros más prometedores.



Los recursos computacionales son suficientes para evaluar cada combinación de hiperparámetros

# Optimización por grillas:

## Librerías de python



### Scikit-learn (`sklearn.model_selection.GridSearchCV`):

Proporciona una implementación fácil de usar de **Grid Search** con validación cruzada, permitiendo una evaluación exhaustiva de cada modelo.



**TensorFlow y Keras** (a través de `keras.wrappers.scikit_learn` para integración con Scikit-learn): Permiten utilizar **Grid Search** con modelos de aprendizaje profundo.

# Optimización por grillas:

## Datos clave



**Eficiencia:** Aunque Grid Search es exhaustivo, puede ser muy ineficiente cuando el espacio de hiperparámetros es grande.



**Valores de Hiperparámetros:** Es crucial elegir un rango y una granularidad adecuados para los valores de los hiperparámetros a explorar.



**Validación Cruzada:** Usar validación cruzada dentro del Grid Search ayuda a obtener una estimación más robusta del rendimiento del modelo.

# Optimización

## GridSearch vs Keras Tuner

| Característica  | Grid Search                                                                                                | Keras Tuner                                                                                                             | Hyperband                                                                                                                   |
|-----------------|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Descripción     | Prueba sistemáticamente todas las combinaciones posibles de hiperparámetros en una cuadrícula predefinida. | Ofrece varias estrategias de búsqueda, incluida Hyperband, para optimizar hiperparámetros.                              | Utiliza asignación de recursos adaptativa y detención temprana para converger rápidamente en un modelo de alto rendimiento. |
| Eficiencia      | Puede ser computacionalmente costoso y lento, especialmente con grandes espacios de búsqueda.              | Más eficiente que Grid Search gracias a métodos como Hyperband y búsqueda bayesiana.                                    | Altamente eficiente al descartar modelos de bajo rendimiento rápidamente.                                                   |
| Uso Recomendado | Espacios de búsqueda pequeños a medianos donde es viable probar todas las combinaciones.                   | Modelos de aprendizaje profundo y cuando se necesita explorar un espacio de hiperparámetros grande de manera eficiente. | Optimización rápida en espacios de búsqueda amplios con presupuestos computacionales limitados.                             |
| Implementación  | Ampliamente soportado por librerías como Scikit-learn.                                                     | Disponible a través de la librería Keras Tuner.                                                                         | Disponible como parte de Keras Tuner y otras librerías de optimización de hiperparámetros.                                  |
| Paralelización  | Soporte limitado para paralelización.                                                                      | Permite la paralelización tanto de datos como de pruebas de hiperparámetros.                                            | Soporte intrínseco para ejecuciones paralelas y distribuidas.                                                               |
| Adaptabilidad   | Limitada; no ajusta su estrategia basada en resultados intermedios.                                        | Flexible; permite definir modelos hiperpersonalizables y adapta la búsqueda basándose en resultados.                    | Muy adaptable; ajusta dinámicamente el número de épocas y recursos basándose en el rendimiento.                             |

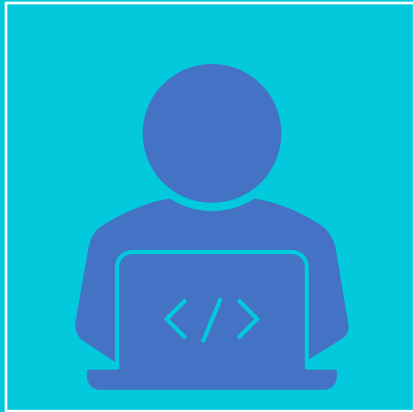
# Optimización por búsqueda aleatoria:

## Conceptos

La búsqueda aleatoria implica seleccionar aleatoriamente valores de hiperparámetros y es más eficaz para identificar valores de hiperparámetros óptimos sin una hipótesis sólida. El proceso de muestreo aleatorio es más eficiente y normalmente devuelve un conjunto de valores óptimos basados en menos iteraciones del modelo. Por ejemplo, en los modelos de aprendizaje profundo, la búsqueda aleatoria puede ayudar a descubrir rápidamente tasas de aprendizaje, tamaños de lotes o arquitecturas de red óptimas.

# Búsqueda aleatoria:

## Librerías de python



Puedes utilizar Random Search con la biblioteca Scikit-learn en Python usando **RandomizedSearchCV**.



Este método es recomendable cuando se busca reducir el tiempo de búsqueda y se tiene un espacio de búsqueda extenso.

# Búsqueda aleatoria:

## Ejemplo de código de python

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
from scipy.stats import randint

# Define el modelo
modelo = RandomForestClassifier()

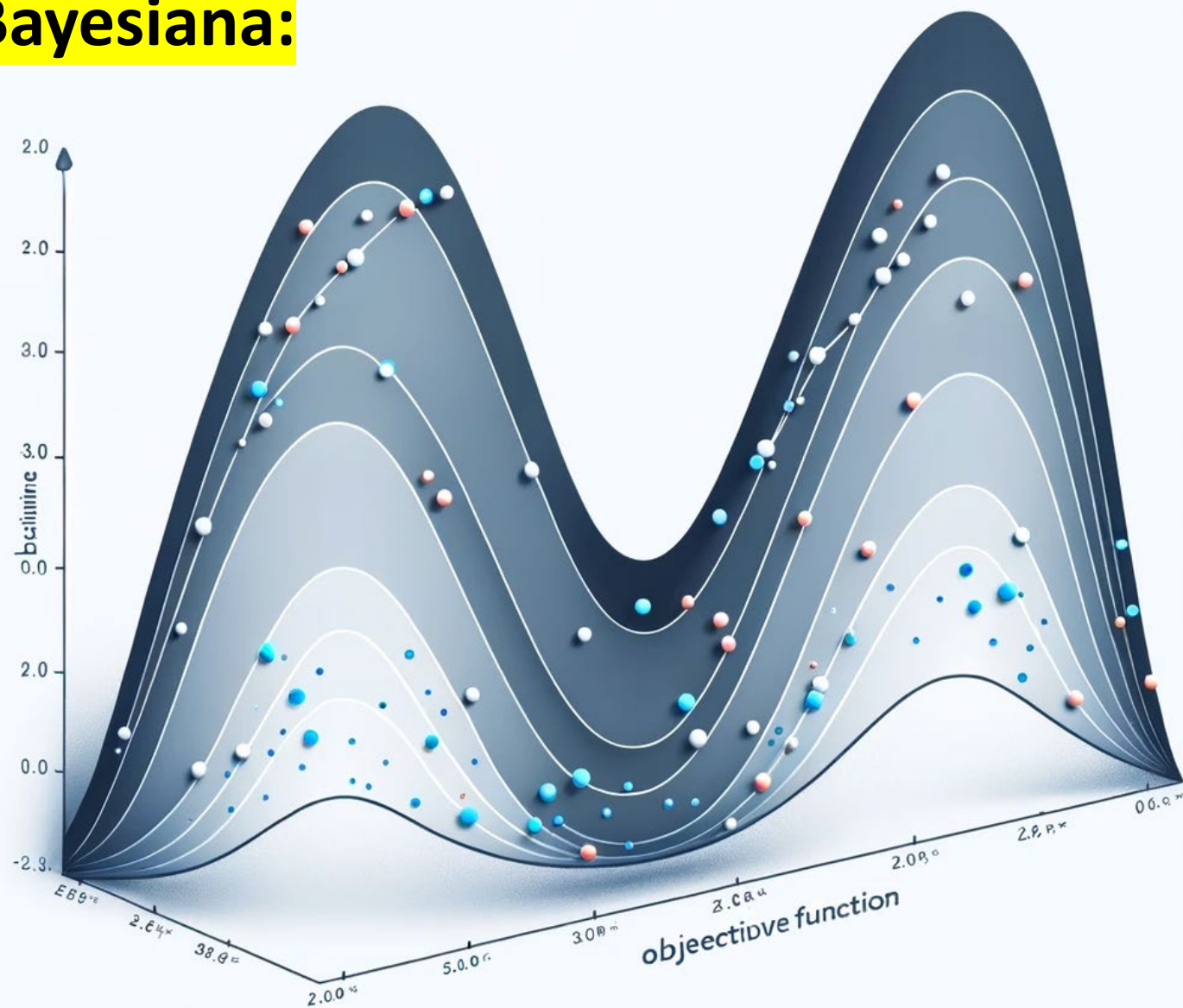
# Espacio de hiperparámetros
param_dist = {'n_estimators': randint(100, 200),
              'max_depth': randint(3, 10)}

# Random Search
random_search = RandomizedSearchCV(model, param_distributions=param_dist, n_iter=50)
random_search.fit(X_train, y_train)
```

# Optimización por búsqueda Bayesiana:

## Conceptos

La búsqueda bayesiana es una técnica avanzada de optimización de hiperparámetros basada en el teorema de Bayes. Opera mediante la construcción de un modelo probabilístico de la función objetivo, conocida como función sustituta, que luego se busca de manera eficiente utilizando una función de adquisición antes de seleccionar muestras candidatas para su evaluación en la función objetivo real.

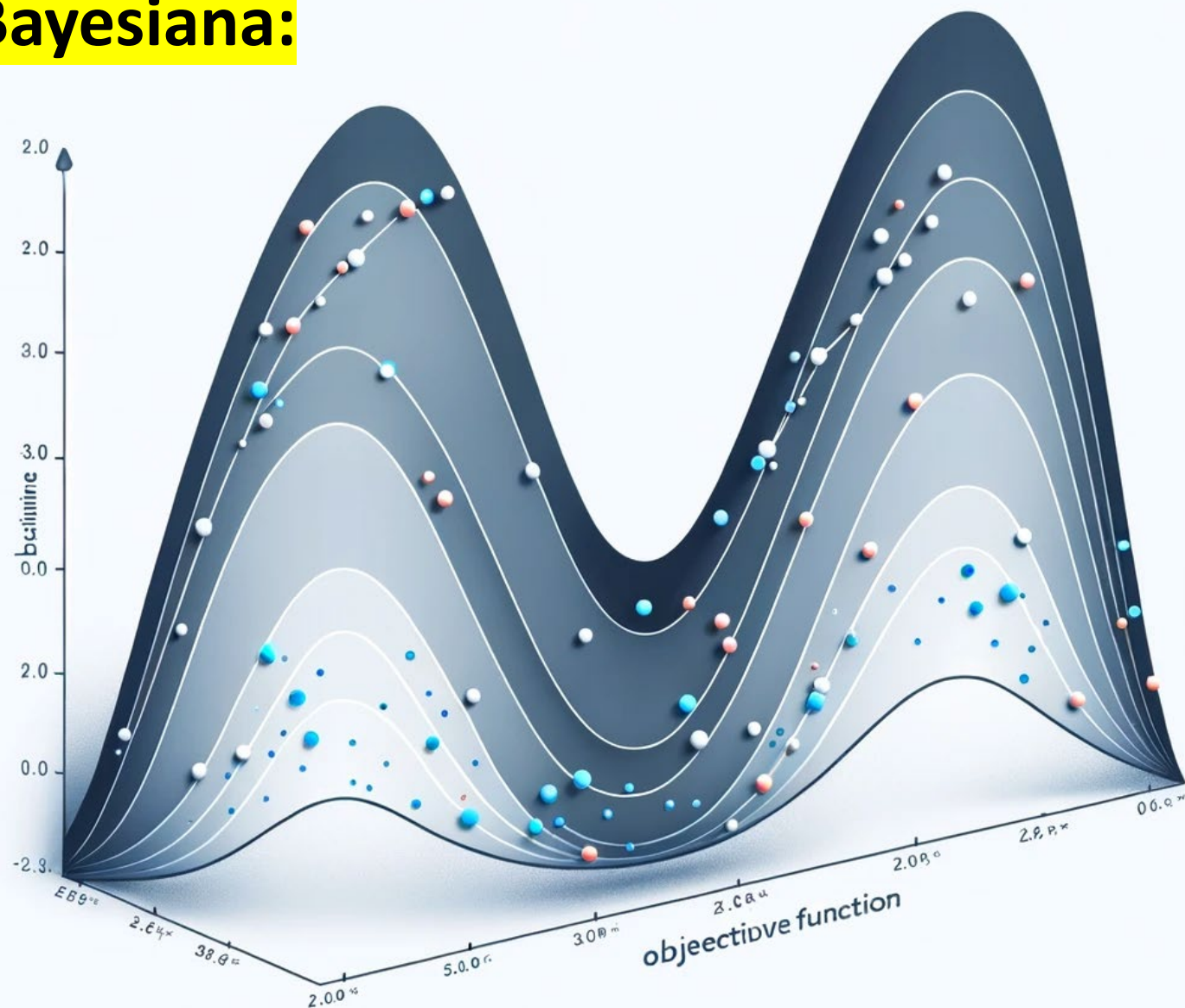


# Optimización por búsqueda Bayesiana:

## Conceptos

La optimización bayesiana se utiliza normalmente en problemas de la forma  $\max_{x \in A} f(x)$ , donde  $A$  es un conjunto de puntos,  $X$  que se basan en menos de 20 dimensiones ( $\mathbb{R}^d, d \leq 20$ ),

y cuya pertenencia puede evaluarse fácilmente. La optimización bayesiana es especialmente ventajosa para problemas en los que es difícil de evaluar debido a su coste computacional. La función objetivo,  $f$ , es continua y adopta la forma de una estructura desconocida, denominada "caja negra". Tras su evaluación, sólo  $f(x)$  se observa y sus derivadas no se evalúan.<sup>7</sup>

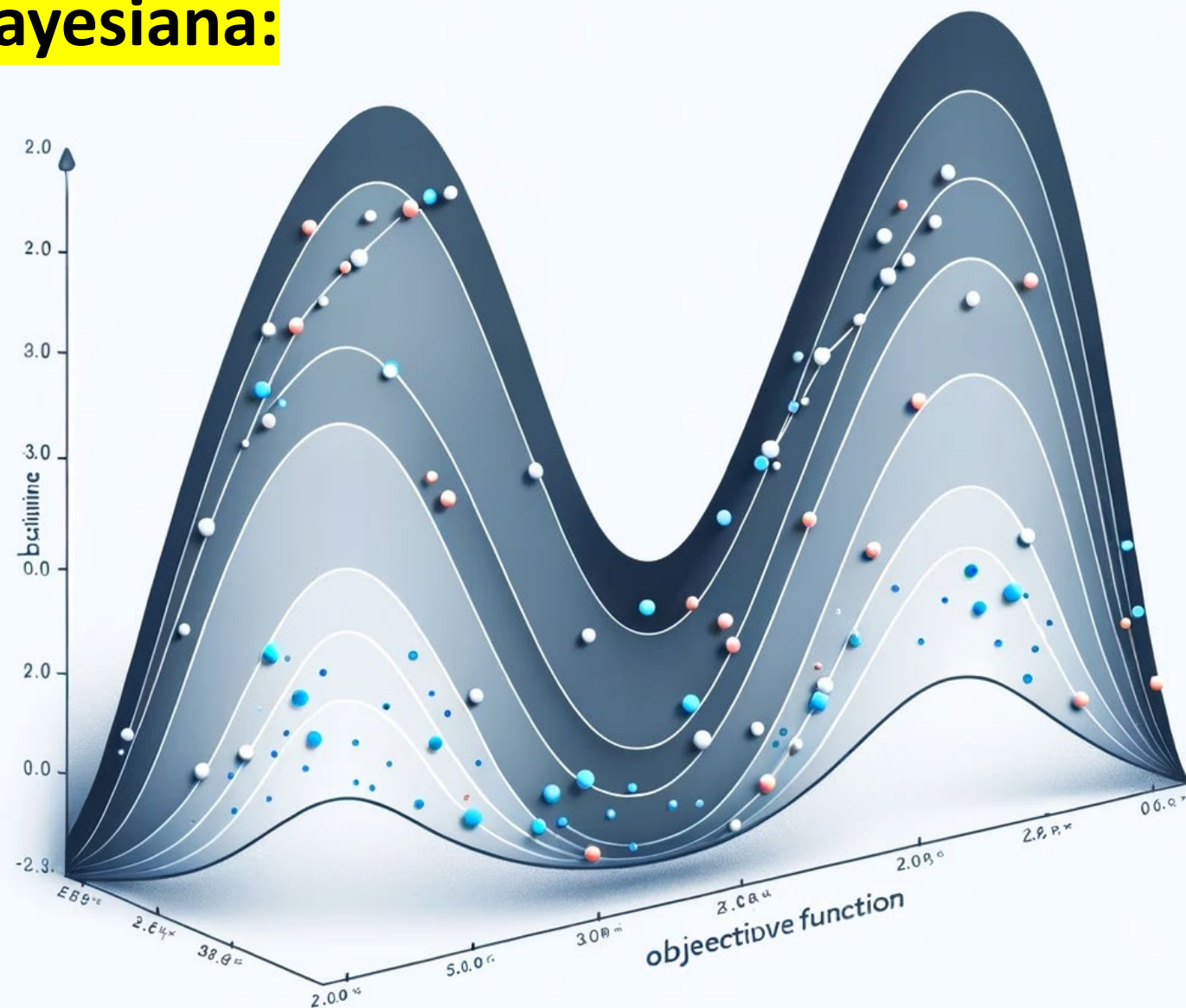


# Optimización por búsqueda Bayesiana:

## Conceptos

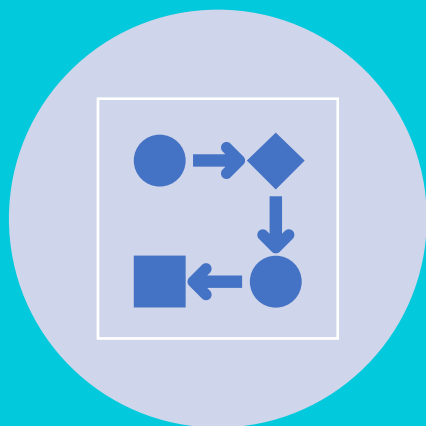
En un modelo de regresión logística, se puede emplear la optimización bayesiana para identificar los parámetros de regularización y las tasas de aprendizaje óptimos.

A diferencia de los métodos de búsqueda aleatoria o de Grilla, que exploran el espacio de hiperparámetros sin tener en cuenta el rendimiento pasado, la optimización bayesiana construye un modelo probabilístico de la función objetivo (el rendimiento del modelo como función de los hiperparámetros) y lo utiliza para predecir cuáles hiperparámetros podrían ofrecer mejoras.



# Optimización Bayesiana

Lo más importante de este método



**EFICIENCIA:** PUEDE ENCONTRAR MEJORES HIPERPARÁMETROS CON MENOS EVALUACIONES, YA QUE UTILIZA EL CONOCIMIENTO PREVIO PARA ELEGIR LOS PRÓXIMOS PUNTOS A EVALUAR.



**MODELO DE PREDICCIÓN:** UTILIZA LOS RESULTADOS DE EVALUACIONES ANTERIORES PARA FORMAR UN MODELO PROBABILÍSTICO DE LA FUNCIÓN OBJETIVO.

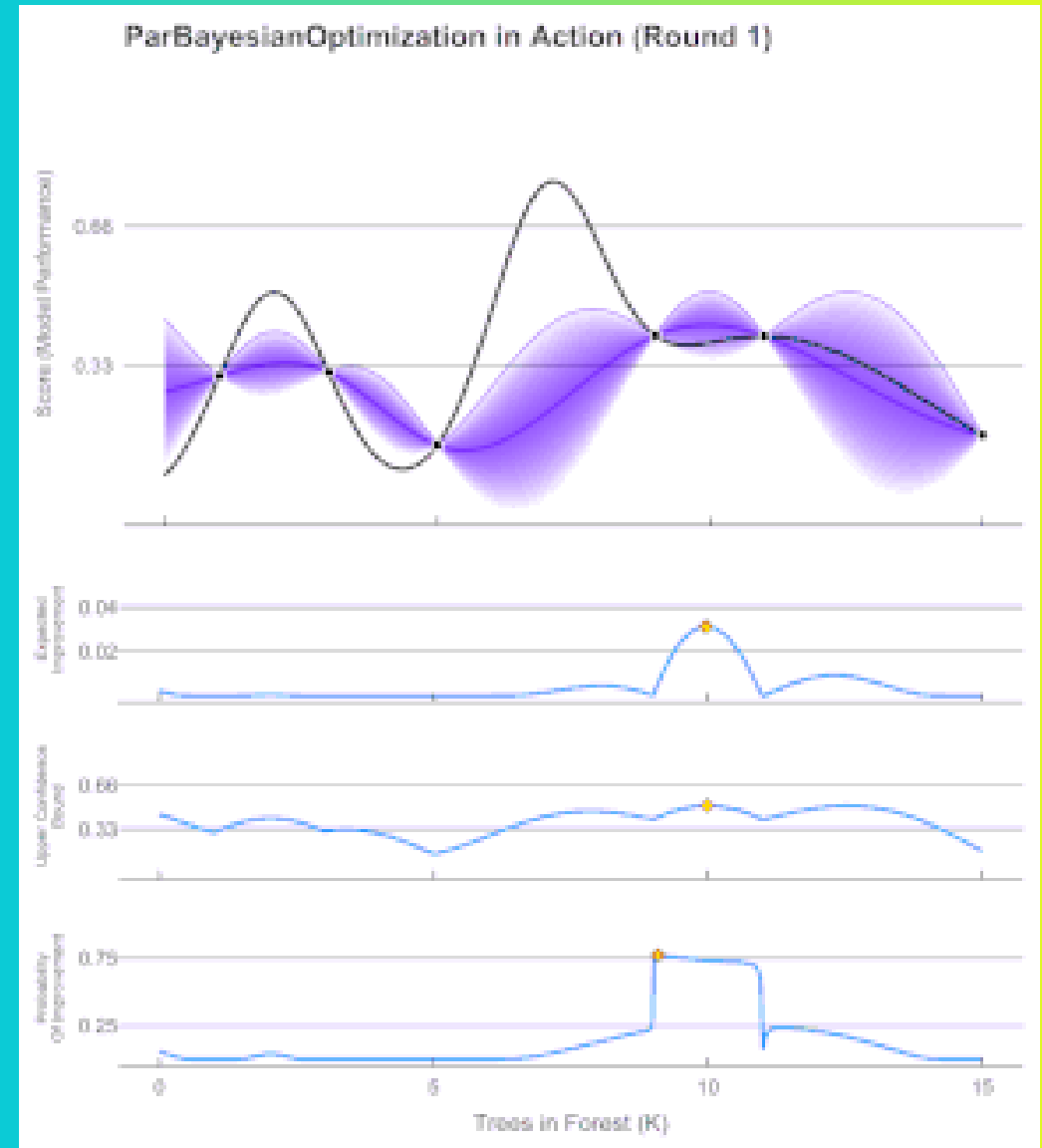


**SELECCIÓN DE PUNTOS:** SELECCIONA EL PRÓXIMO PUNTO A EVALUAR AL EQUILIBRAR LA EXPLORACIÓN (PROBAR PARTES NO EXPLORADAS DEL ESPACIO) Y LA EXPLOTACIÓN (REFINAR ALREDEDOR DE LOS MEJORES HIPERPARÁMETROS ENCONTRADOS).

# Optimización Bayesiana

## ¿Cuándo aplicarlo?

Es particularmente útil cuando las evaluaciones de la función objetivo son costosas (en tiempo o recursos) y se tiene un número limitado de evaluaciones. También es eficaz cuando no se conoce mucho sobre la forma de la función objetivo o cuando el espacio de hiperparámetros es grande.



# Optimización Bayesiana

## Librerías en Python

**Scikit-Optimize (skopt):**  
Proporciona una interfaz sencilla para la optimización bayesiana.

**Hyperopt:** Otra popular biblioteca para optimización bayesiana con soporte para paralelización y espacios de hiperparámetros complejos.

**Optuna:** Una biblioteca de optimización de hiperparámetros con una API intuitiva y características como el pruning automático.

# Optimización Bayesiana

Modelos probabilísticos que se pueden incluir

## Procesos Gaussianos (Gaussian Processes - GP):

Los Procesos Gaussianos son un modelo probabilístico que asume que las observaciones de la función objetivo provienen de una distribución gaussiana multivariante. Un GP es completamente especificado por su función media (a menudo asumida como cero en la práctica de optimización) y su función de covarianza (o kernel), que define cómo varían las salidas de la función en relación con los puntos de entrada. Los GP son útiles porque proporcionan tanto una estimación de la función objetivo como una medida de incertidumbre en cada punto del espacio de hiperparámetros. Son especialmente buenos para modelar funciones suaves y son ampliamente usados debido a su flexibilidad y capacidad para proporcionar estimaciones de incertidumbre.

# Optimización Bayesiana

Modelos probabilísticos que se pueden incluir



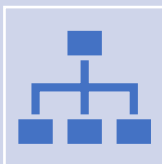
## 2. Árboles de Regresión Bayesianos (Bayesian Regression Trees - BART)



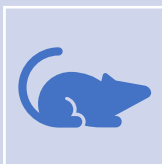
BART es un enfoque que utiliza árboles de decisión como base para construir un modelo probabilístico. Consiste en sumar las predicciones de muchos árboles de decisión para formar la estimación final. Cada árbol está restringido en su complejidad, lo que hace que el modelo completo sea menos propenso al sobreajuste. BART captura de manera efectiva las relaciones no lineales y las interacciones entre variables

# Optimización Bayesiana

Modelos probabilísticos que se pueden incluir



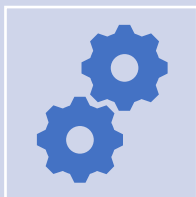
## 3. Estimadores de Parzen (Tree-structured Parzen Estimator - TPE)



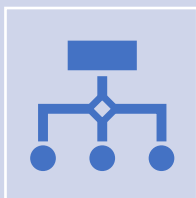
TPE es un algoritmo que modela la función objetivo con dos densidades de probabilidad: una para los valores que conducen a mejores resultados y otra para el resto. A diferencia de los Procesos Gaussianos, TPE no asume una forma específica para la función objetivo. Selecciona los próximos puntos a evaluar al preferir regiones del espacio donde la relación entre estas dos densidades es más favorable. TPE puede ser más eficiente que los GP cuando el espacio de búsqueda es grande.

# Optimización Bayesiana

Modelos probabilísticos que se pueden incluir



## 4. Optimización basada en modelos secuenciales (Sequential Model-Based Optimization - SMBO)



Aunque SMBO es más una metodología que un modelo específico, utiliza modelos probabilísticos (incluidos GP, TPE, y otros) para secuencialmente construir un modelo de la función objetivo basado en los resultados de evaluaciones anteriores. El objetivo es utilizar este modelo para seleccionar el punto más prometedor para la próxima evaluación, equilibrando entre explorar nuevas regiones del espacio de búsqueda y explotar las áreas conocidas que tienen un alto rendimiento.

# Optimización Bayesiana

Modelos probabilísticos que se pueden incluir

| Modelo                                        | Cuando usarlo                                                                                     | Ventaja                                                            | Biblioteca en Python          | Función en Python                                                        | Desventaja                                                                      |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|-------------------------------|--------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <b>Procesos Gaussianos (GP)</b>               | Pequeñas a medianas dimensiones del espacio de búsqueda; funciones suaves.                        | Estimación precisa y medida de incertidumbre.                      | scikit-learn, GPy, GPflow     | GaussianProcessRegressor en scikit-learn                                 | Puede ser computacionalmente costoso en espacios de alta dimensión.             |
| <b>Árboles de Regresión Bayesianos (BART)</b> | Datos con relaciones complejas y no lineales; interacciones desconocidas.                         | Capta eficazmente no linealidades e interacciones entre variables. | pybart                        | No directamente disponible, se utiliza a través de la biblioteca pybart. | Puede ser más desafiante de interpretar en comparación con modelos más simples. |
| <b>Estimadores de Parzen (TPE)</b>            | Espacios de búsqueda grandes; optimización de hiperparámetros.                                    | Eficiente en espacios de alta dimensión.                           | hyperopt                      | fmin con tpe.suggest                                                     | Puede no capturar dependencias complejas tan efectivamente como los GP.         |
| <b>SMBO</b>                                   | Cuando las evaluaciones de la función objetivo son costosas; problemas generales de optimización. | Flexible; puede incorporar diferentes modelos subyacentes.         | scikit-optimize, SMAC, optuna | Varía según la biblioteca, e.g., gp_minimize en scikit-optimize          | La eficacia depende del modelo subyacente y del ajuste de hiperparámetros.      |

# Optimización Bayesiana

## Datos clave

La optimización bayesiana es especialmente útil para optimizar hiperparámetros de modelos complejos y costosos de entrenar.

La elección del modelo probabilístico (por ejemplo, procesos gaussianos, árboles de Parzen) puede afectar el rendimiento de la optimización.

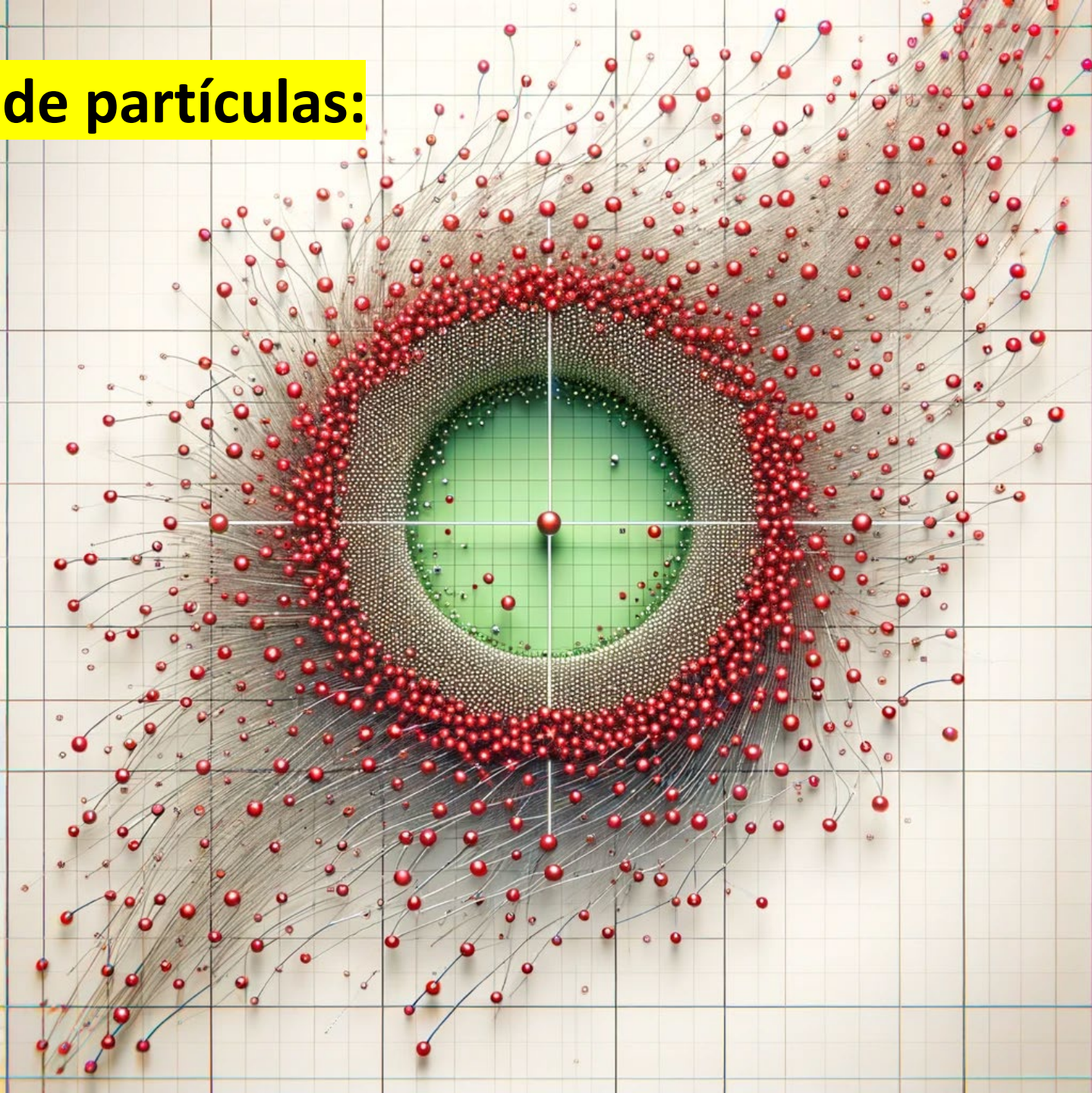
Aunque es más eficiente que la búsqueda en Grilla o aleatoria, la complejidad del modelo probabilístico puede aumentar el tiempo de computación por iteración.

# Optimización por enjambre de partículas:

## Conceptos

Particle Swarm Optimization (PSO) es una técnica de optimización inspirada en la naturaleza que simula el comportamiento social de un grupo de organismos, como aves o peces, en busca de una solución. PSO se utiliza a menudo para problemas de optimización continua, incluido el ajuste de hiperparámetros en modelos de aprendizaje automático.

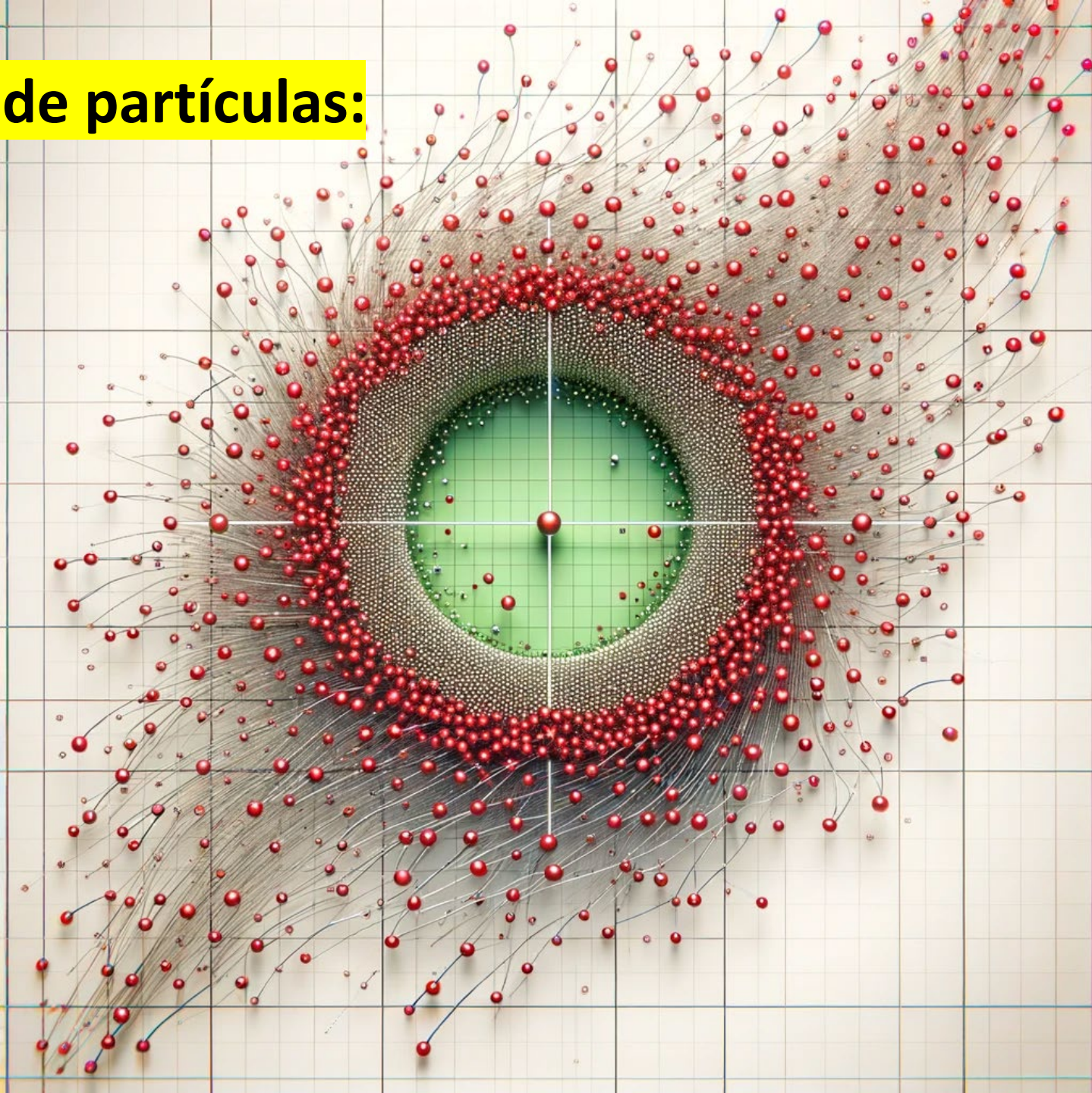
En este método, cada partícula representa una solución potencial en el espacio de búsqueda, y las partículas actualizan iterativamente sus posiciones en función de su mejor solución y de la mejor solución encontrada por todo el enjambre.



# Optimización por enjambre de partículas:

## Conceptos

Por ejemplo, al optimizar hiperparámetros para una red neuronal, se puede emplear PSO para explorar el espacio de búsqueda de tasas de aprendizaje, funciones de activación y el número de capas ocultas. Al actualizar iterativamente las posiciones de las partículas, PSO converge hacia un óptimo global, proporcionando un conjunto de hiperparámetros óptimos para el modelo de aprendizaje automático.



# Optimización PSO

## Como funciona



**Inicialización:** El algoritmo comienza con un enjambre de partículas (soluciones candidatas) distribuidas aleatoriamente en el espacio de búsqueda.



**Evaluación:** Cada partícula evalúa su posición actual en el espacio de búsqueda utilizando la función objetivo.



**Actualización de velocidades y posiciones:** Basándose en sus experiencias personales y las de sus vecinos, las partículas ajustan sus velocidades y, por lo tanto, sus posiciones hacia las mejores soluciones encontradas.



**Repetición:** Los pasos 2 y 3 se repiten hasta que se cumpla un criterio de terminación, como alcanzar un número máximo de iteraciones o lograr una convergencia satisfactoria.

# Optimización PSO

## Su importancia

**Influencia social:** PSO simula el aprendizaje social y el comportamiento colectivo para dirigir a las partículas hacia las mejores soluciones.

**Flexibilidad:** Funciona bien para una amplia gama de problemas sin necesidad de gradientes o derivadas de la función objetivo.

**Paralelización:** El algoritmo se presta naturalmente a la paralelización, ya que las partículas pueden evaluarse de manera independiente.

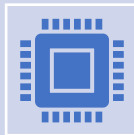
# Optimización PSO

## ¿Cuándo aplicarlo?

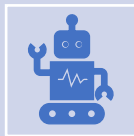
PSO es especialmente útil para problemas de optimización donde las soluciones pueden ser codificadas como vectores de números reales o enteros, y es particularmente efectivo en espacios de búsqueda de alta dimensión donde los métodos basados en gradientes pueden no ser aplicables. Es adecuado para problemas de optimización continua y también ha sido adaptado para problemas discretos.

# Optimización PSO

## Librerías en Python



**PySwarms:** Es una biblioteca de optimización moderna que implementa PSO en Python, es fácil de usar y altamente flexible.



**DEAP (Distributed Evolutionary Algorithms in Python):** Aunque es más conocida por sus capacidades en algoritmos genéticos, DEAP también ofrece soporte para PSO.

# Optimización PSO

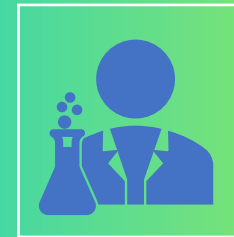
## Claves



**PSO** no garantiza encontrar el óptimo global, especialmente en paisajes de búsqueda complejos con múltiples mínimos locales.



La elección de los parámetros del PSO (como los coeficientes cognitivos y sociales, y el factor de inercia) puede tener un gran impacto en el rendimiento del algoritmo.

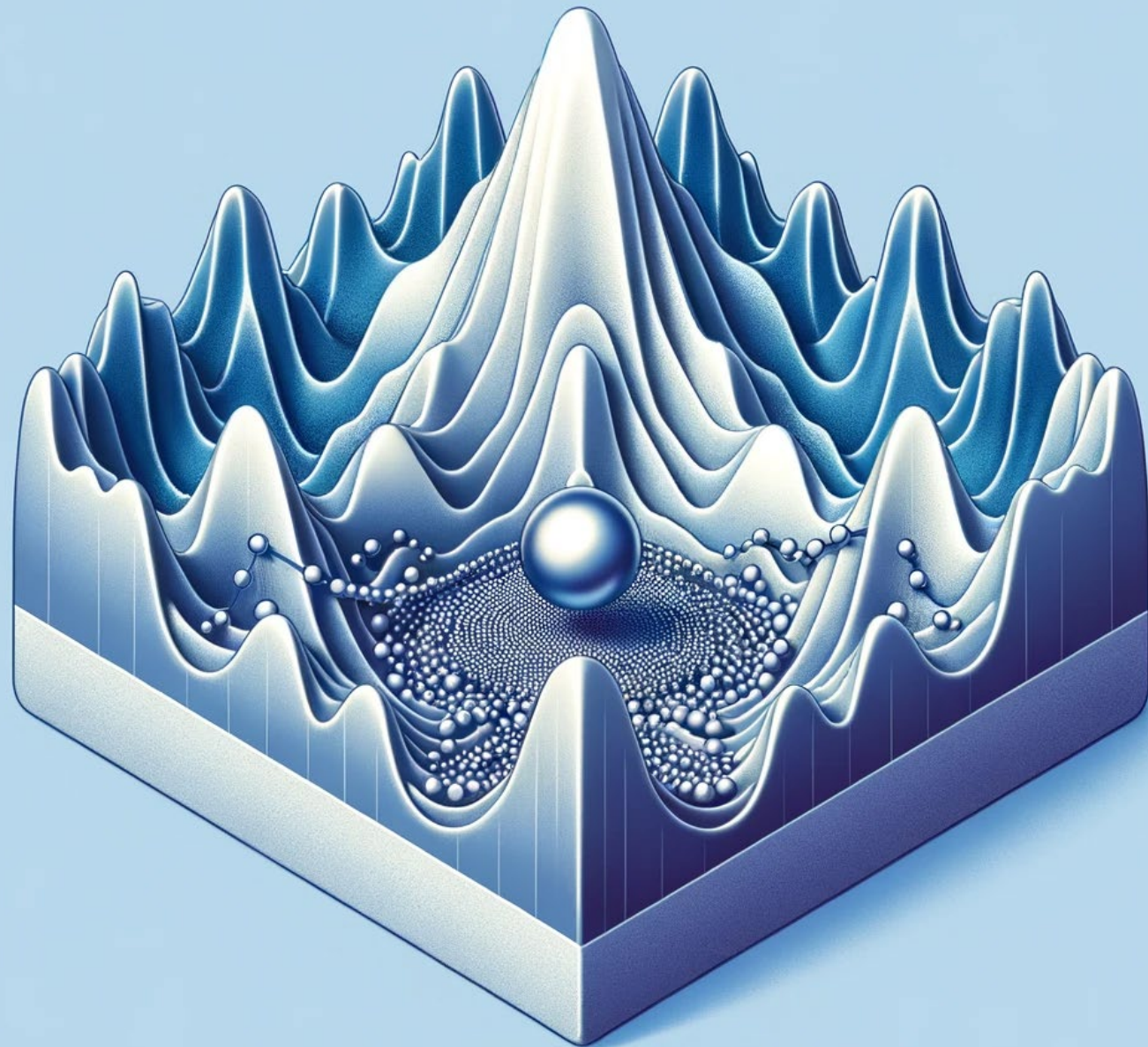


PSO es altamente dependiente del ajuste de parámetros y puede requerir experimentación para lograr los mejores resultados.

# Simulated Annealing (SA):

## Conceptos

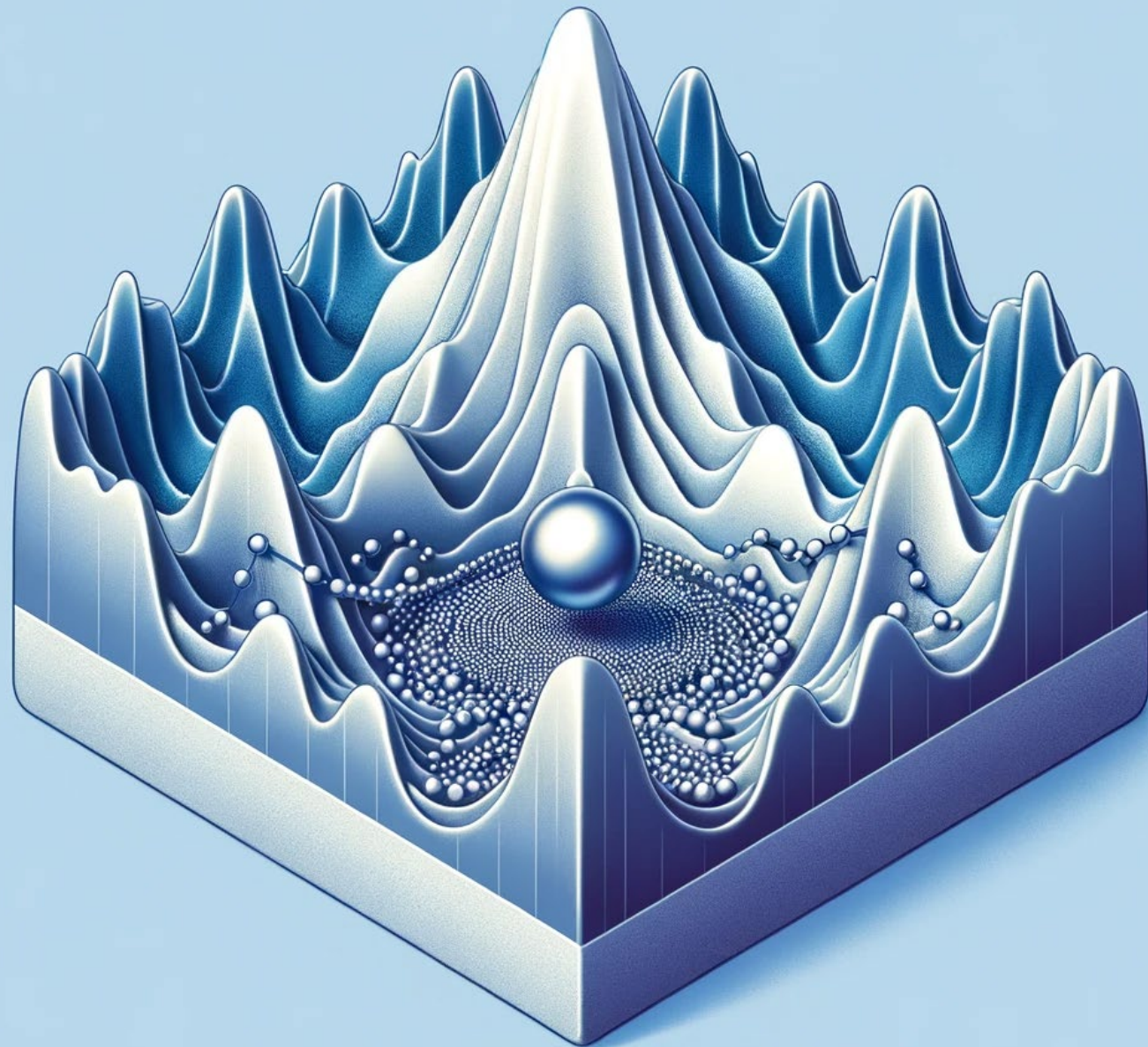
El recocido simulado (SA) es un algoritmo de optimización inspirado en el proceso de recocido en metalurgia, donde un material se enfría lentamente para reducir los defectos y mejorar su estructura. El algoritmo funciona reduciendo gradualmente la probabilidad de aceptar peores soluciones a medida que avanza la búsqueda, lo que le permite escapar de los mínimos locales y converger hacia un óptimo global.



# Simulated Annealing (SA):

## Conceptos

Cuando se aplica a la optimización de hiperparámetros en modelos de aprendizaje automático, como un clasificador de bosque aleatorio, SA se puede utilizar para explorar el espacio de búsqueda de la cantidad de árboles, la profundidad máxima y las muestras mínimas por hoja. Al disminuir gradualmente el parámetro de temperatura, el algoritmo se vuelve más selectivo al aceptar nuevas soluciones y, en última instancia, produce un conjunto óptimo de hiperparámetros.



# Simulated Annealing (SA)

## ¿Como funciona?



**Calentamiento inicial:** Se comienza con una temperatura alta que permite al algoritmo explorar ampliamente el espacio de soluciones.



**Enfriamiento:** La temperatura disminuye gradualmente según un horario de enfriamiento, reduciendo la probabilidad de aceptar soluciones peores. Esto ayuda a converger gradualmente hacia un óptimo.



**Búsqueda de soluciones:** Se selecciona una solución aleatoria cerca de la solución actual. Si la nueva solución es mejor, se adopta. Si es peor, se puede adoptar con una probabilidad que depende de la diferencia de calidad entre la solución actual y la nueva, y de la temperatura actual.



**Condiciones de parada:** El proceso se repite hasta que se cumplan ciertas condiciones, como alcanzar un número máximo de iteraciones o una temperatura mínima.

# Simulated Annealing (SA)

Lo más importante del método



**Escape de óptimos locales:** SA puede escapar de óptimos locales debido a la posibilidad de aceptar soluciones peores al principio.



**Horario de enfriamiento:** La forma en que se reduce la temperatura es crucial y puede afectar significativamente la eficacia del algoritmo.

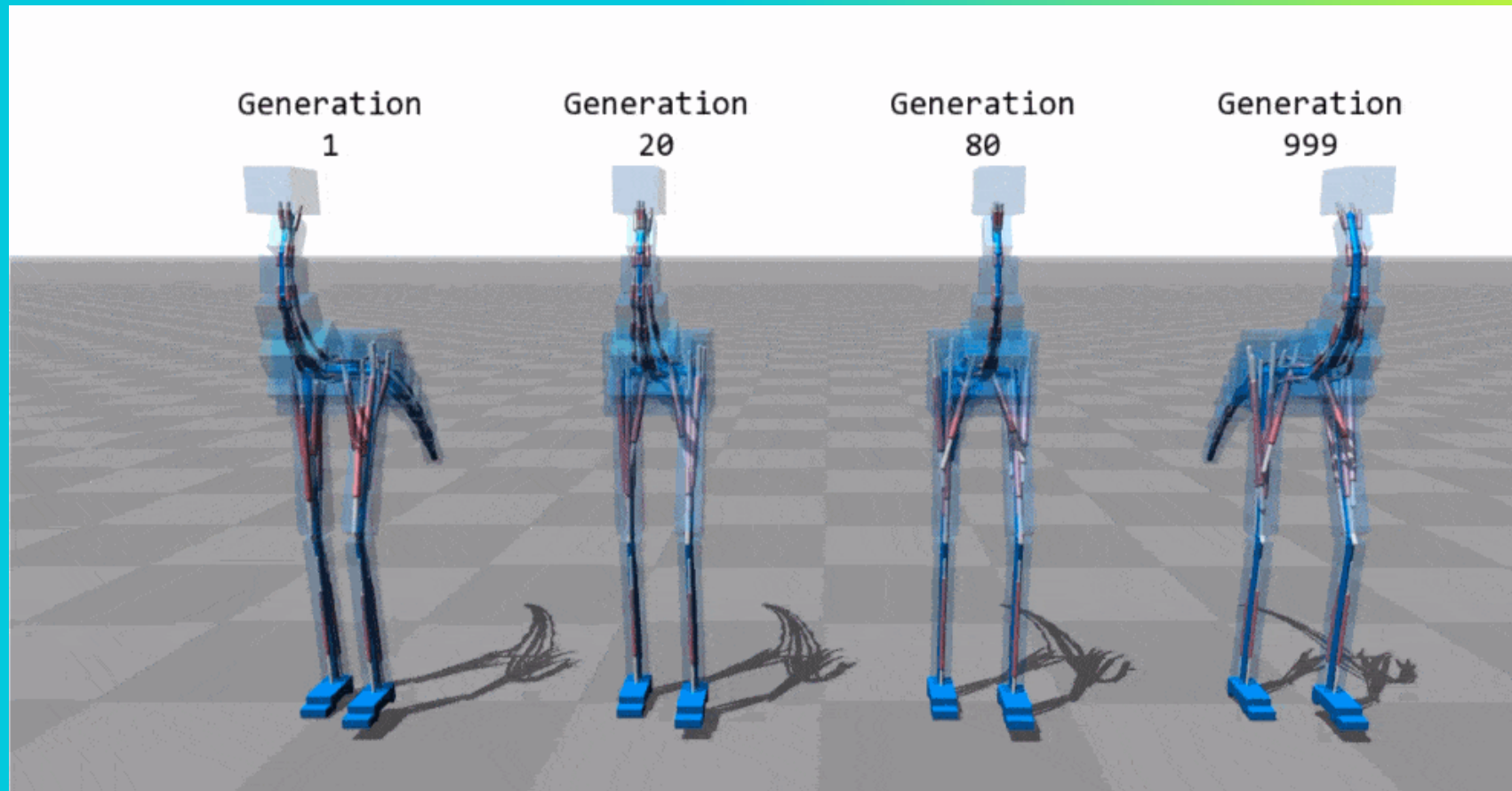
# Simulated Annealing (SA)

## Cuando Aplicarlo

**SA** es particularmente útil en problemas de optimización donde el espacio de soluciones es grande y contiene muchos óptimos locales. Es adecuado para problemas donde otros algoritmos como la búsqueda en cuadrícula o la búsqueda aleatoria se atascan en óptimos locales.

# Algoritmos genéticos

## Conceptos



Los **AG** se emplean con frecuencia para generar soluciones de alta calidad para problemas de optimización y búsqueda basándose en operadores de inspiración biológica como la mutación, el cruce y la selección.

# Algoritmos genéticos

## Lo más importante del método

**Diversidad de la población:**  
Fundamental para explorar eficazmente el espacio de búsqueda.

**Selección de individuos según aptitud:** Asegura que las características deseables se transmitan a las generaciones futuras.

**Balance entre exploración y explotación:** Crucial para encontrar soluciones óptimas sin quedar atrapado en mínimos locales

# Algoritmos genéticos

## Aplicabilidad

Los algoritmos genéticos son particularmente útiles en problemas de optimización complejos donde otros métodos, como la búsqueda exhaustiva o los métodos basados en gradientes, no son prácticos o no aplican, como en los problemas de optimización combinatoria y en aquellos donde el espacio de búsqueda es discontinuo o muy irregular.

# Algoritmos genéticos

## Librerías en Python



**DEAP (Distributed Evolutionary Algorithms in Python):** Es una biblioteca muy utilizada que permite implementar algoritmos genéticos con un alto grado de personalización.



**ga (Genetic Algorithm):** Una biblioteca de Python simple para algoritmos genéticos.



**Pyevolve:** Otra biblioteca que ofrece implementaciones de algoritmos genéticos.

# Resultados de optimización

## Naturaleza estocástica de los algoritmos

Tanto PSO, SA, RandomSearch y la Optimización Bayesiana son intrínsecamente estocásticos.

**PSO:** En la Optimización por Enjambre de Partículas, la exploración del espacio de soluciones se realiza mediante un conjunto de "partículas" que se mueven por el espacio de búsqueda. Estas partículas ajustan sus trayectorias basándose tanto en su mejor posición personal encontrada como en la mejor posición encontrada por el enjambre. La aleatoriedad juega un papel crucial en cómo se ajustan estas trayectorias, lo que significa que diferentes ejecuciones pueden explorar el espacio de soluciones de manera diferente, incluso si la semilla del generador de números aleatorios se fija.

**Optimización Bayesiana:** Este método construye un modelo probabilístico del espacio de búsqueda y utiliza este modelo para predecir las regiones del espacio que probablemente lleven a mejores resultados. La elección de puntos para evaluar se guía por una función de adquisición, que equilibra la exploración frente a la explotación basada en el conocimiento actual. Dado que este conocimiento se actualiza con cada evaluación y que el proceso incluye componentes aleatorios (p. ej., en la inicialización), diferentes ejecuciones pueden identificar diferentes conjuntos de hiperparámetros óptimos.

# Resultados de optimización

## Consistencia en los resultados

La consistencia en la calidad de los resultados a pesar de las diferencias en los hiperparámetros específicos se debe a que los algoritmos están identificando soluciones que son esencialmente equivalentes en términos de rendimiento, dentro de las peculiaridades del espacio de solución. Esto es especialmente común en problemas de aprendizaje automático, donde muchos modelos pueden ser muy robustos a variaciones en ciertos hiperparámetros dentro de rangos razonables

# Resultados de optimización

## ¿Cómo Manejar la Variabilidad?

Fijar la semilla del  
generador de  
números  
aleatorios

Documentar los  
hiperparámetros  
específicos

Considerar el uso  
de promedios y  
varianzas

<https://www.vantage-ai.com/en/blog/bayesian-optimization-for-quicker-hyperparameter-tuning>

<https://towardsdatascience.com/a-conceptual-explanation-of-bayesian-model-based-hyperparameter-optimization-for-machine-learning-b8172278050f>

# Resultados de optimización

## Propiedades del espacio de soluciones

En muchos problemas de optimización, especialmente en aquellos con espacios de alta dimensión y complejos, es común que existan múltiples conjuntos de hiperparámetros que ofrecen rendimientos muy similares. Esto se debe a varias razones:

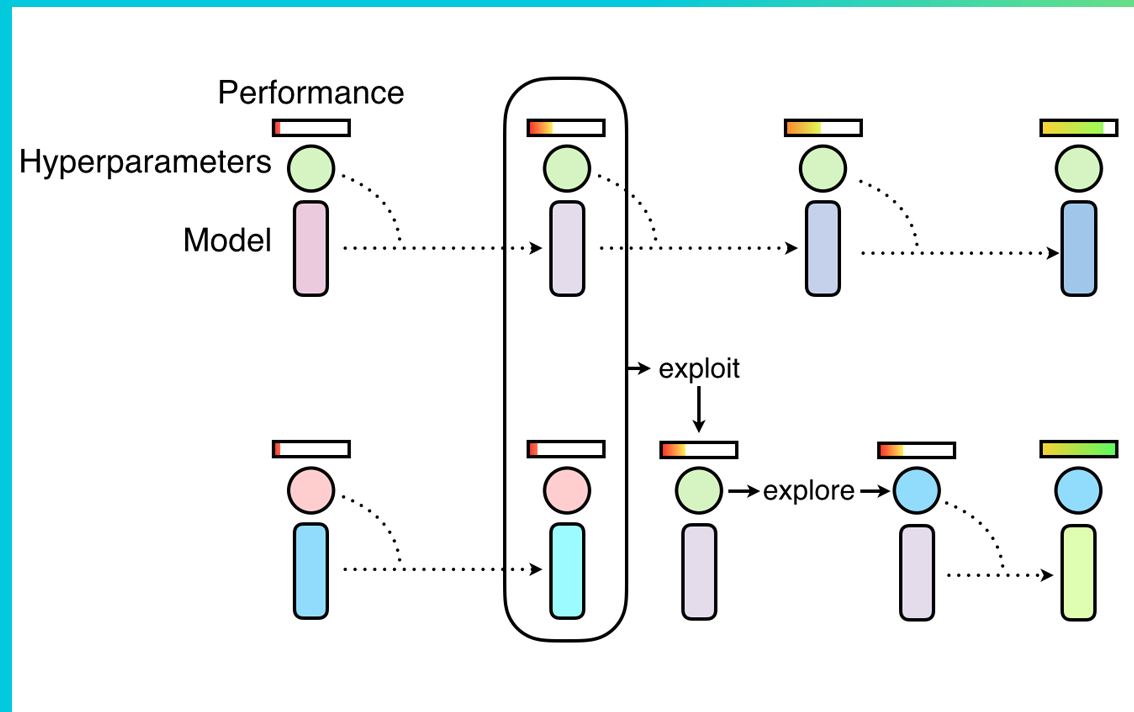
### Existencia de Mínimos Locales

**Superficies de Error Planas:** En ciertas regiones del espacio de búsqueda, pequeñas variaciones en los hiperparámetros pueden no resultar en cambios significativos en el rendimiento del modelo. Esto significa que diferentes conjuntos de hiperparámetros dentro de estas regiones pueden ofrecer rendimientos prácticamente idénticos.

# Entrenamiento basado en la población (PBT)

## Conceptos

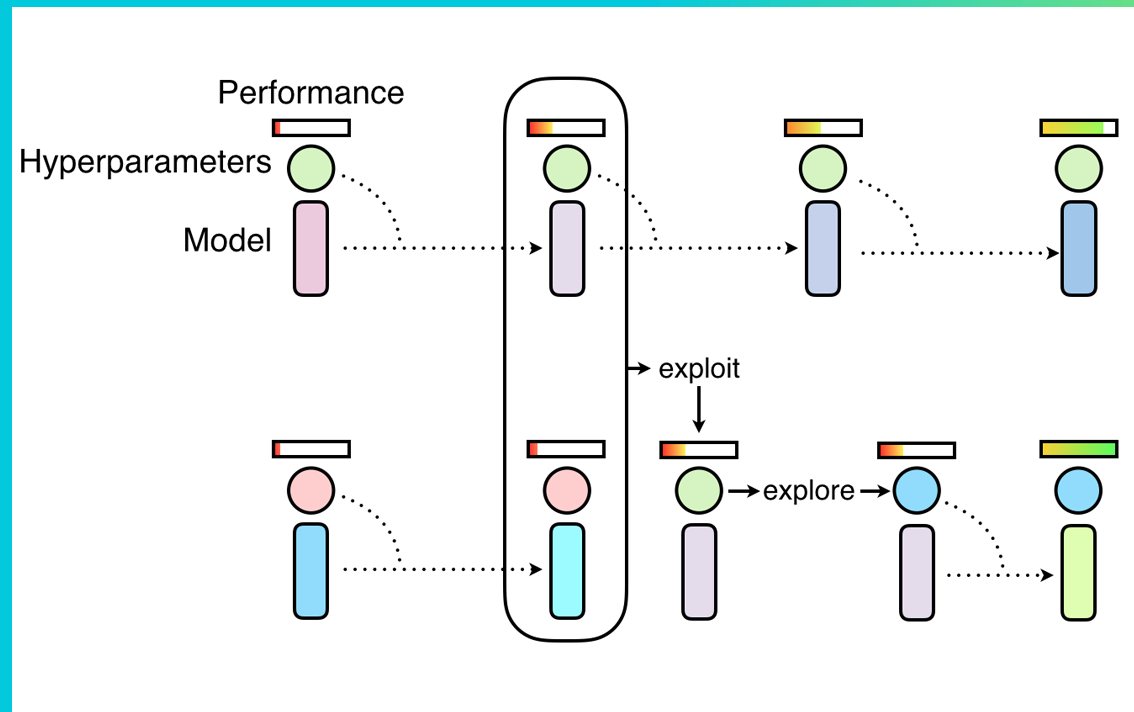
El entrenamiento basado en la población (PBT) es una técnica de optimización para descubrir parámetros e hiperparámetros, basándose en métodos de búsqueda paralelos y métodos de optimización secuencial. Utiliza el intercambio de información entre una población de procesos de optimización que se ejecutan simultáneamente y permite la transferencia en línea de parámetros e hiperparámetros entre miembros de la población en función de su desempeño.



# Entrenamiento basado en la población (PBT)

## Conceptos

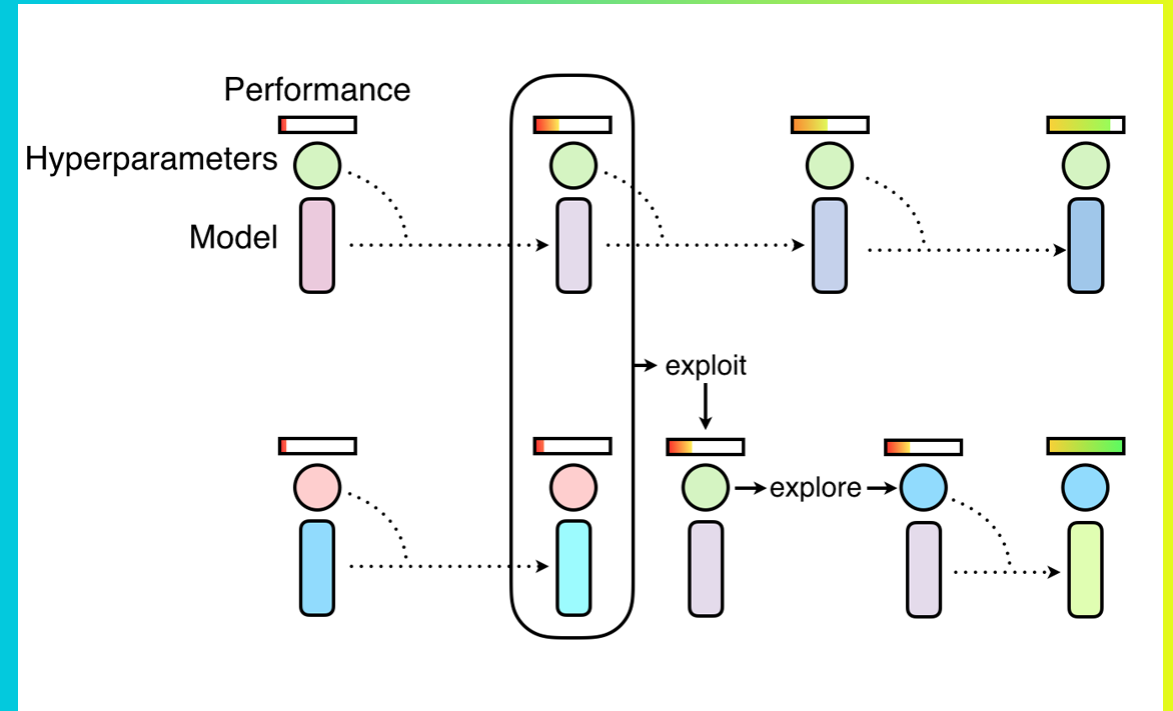
Combina ideas de los algoritmos genéticos y el entrenamiento hiperparamétrico para mejorar continuamente una población de modelos (o hiperparámetros) a lo largo del tiempo. PBT ajusta dinámicamente los hiperparámetros durante el entrenamiento del modelo, lo que permite que los modelos "hereden" hiperparámetros de modelos de mejor desempeño y exploren el espacio de hiperparámetros de manera más eficiente.



# Entrenamiento basado en la población (PBT)

## Conceptos

PBT se puede utilizar para optimizar varios hiperparámetros, como tasas de aprendizaje, tasas de abandono y tamaños de capa. Además, a diferencia de la mayoría de los otros esquemas de adaptación, este método puede realizar una adaptación de hiperparámetros en línea, lo que puede ser crucial en problemas con dinámicas de aprendizaje altamente no estacionarias, como entornos de aprendizaje por refuerzo. PBT es descentralizado y asíncrono, aunque también puede ejecutarse semi-serialmente o con sincronía parcial si existen restricciones presupuestarias.



# Entrenamiento basado en la población (PBT)

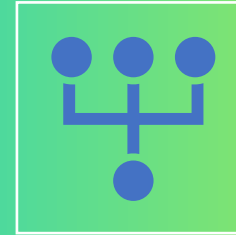
## Lo más importante



**Evolución de la Población:** A diferencia del entrenamiento estándar, donde los hiperparámetros se mantienen fijos, PBT permite que los hiperparámetros evolucionen basándose en el desempeño relativo de los modelos en la población.



**Exploración y Explotación:** PBT equilibra automáticamente entre la exploración de nuevos hiperparámetros y la explotación de hiperparámetros conocidos que funcionan bien, ajustando dinámicamente los hiperparámetros durante el entrenamiento.



**Paralelización y Eficiencia:** Al entrenar una población de modelos en paralelo, PBT puede ser muy eficiente en términos de tiempo y recursos computacionales, especialmente en sistemas con múltiples GPUs o en la nube.

# Entrenamiento basado en la población (PBT)

## Cuando aplicarlo

**PBT** es particularmente útil en escenarios donde el entrenamiento es costoso o lento, como en el caso de modelos de aprendizaje profundo grandes. Es adecuado para:

---

Optimización de hiperparámetros en tareas de aprendizaje automático y aprendizaje profundo.

---

Problemas donde la relación entre hiperparámetros y rendimiento del modelo no es bien entendida o es altamente no lineal.

---

Entornos donde se dispone de recursos computacionales para entrenar múltiples modelos en paralelo.

---

# Entrenamiento basado en la población (PBT)

## Datos clave



PBT requiere una implementación personalizada que maneje la evolución de la población de modelos y sus hiperparámetros. Actualmente, no hay una implementación estándar de PBT en las bibliotecas de aprendizaje automático más populares como scikit-learn o TensorFlow.



El enfoque es más complejo que las técnicas de optimización de hiperparámetros tradicionales, pero puede ofrecer mejoras significativas en rendimiento y eficiencia.



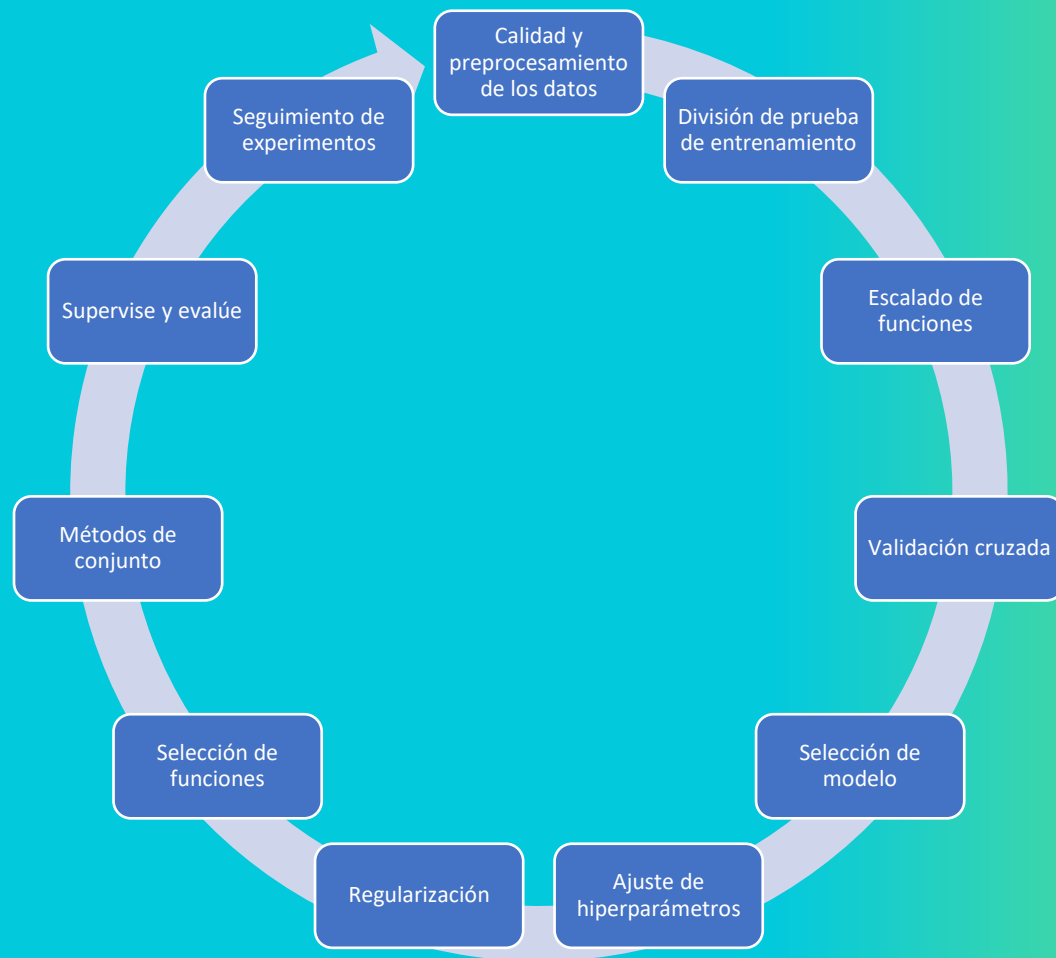
PBT es más adecuado para entornos de investigación y producción donde se dispone del hardware necesario para entrenar múltiples modelos en paralelo.

# Mejores prácticas en optimización de ML

Definición y conceptos básicos

# Mejores practicas

## Datos clave



# Tasa de aprendizaje

## Datos clave

La tasa de aprendizaje es uno de los hiperparámetros más críticos en el entrenamiento de modelos de aprendizaje automático, especialmente en el aprendizaje profundo. Controla la magnitud de los ajustes que se hacen a los parámetros del modelo en respuesta al error calculado en cada iteración del entrenamiento. Elegir una tasa de aprendizaje adecuada es fundamental para asegurar un entrenamiento eficiente y efectivo.

# Importancia de la Tasa de aprendizaje

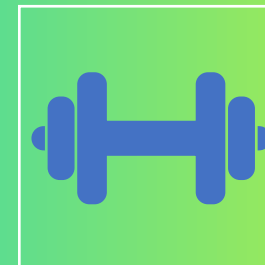
## Datos Clave



**Convergencia:** Una tasa de aprendizaje demasiado alta puede hacer que el entrenamiento oscile o diverja, mientras que una tasa demasiado baja puede llevar a una convergencia muy lenta hacia el mínimo de la función de pérdida.



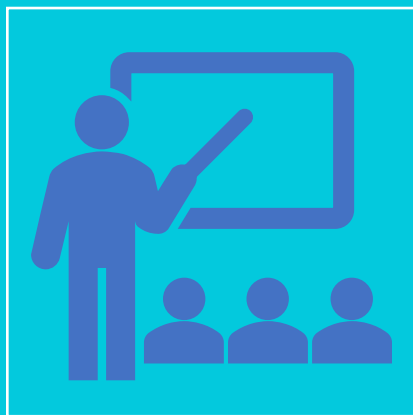
**Precisión del modelo:** Ajustar correctamente la tasa de aprendizaje puede ayudar a encontrar un mínimo global (o un buen mínimo local) de la función de pérdida, lo cual es crucial para el rendimiento del modelo.



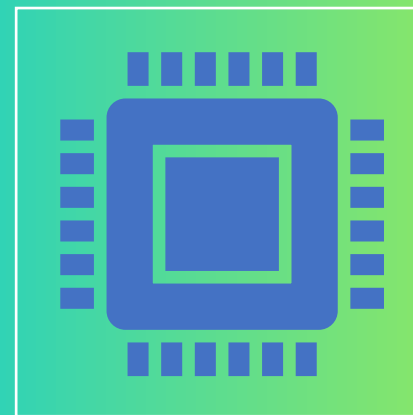
**Eficiencia en el entrenamiento:** Una tasa de aprendizaje óptima permite un entrenamiento más rápido, ahorrando tiempo y recursos computacionales.

# Importancia de la Tasa de aprendizaje

## Ajustes dinamicos



**Decaimiento de la tasa de aprendizaje:** Reducir gradualmente la tasa de aprendizaje a lo largo del tiempo o en respuesta a la falta de mejora en el rendimiento del modelo.



**Métodos adaptativos:** Algoritmos como **Adagrad**, **RMSprop** y **Adam** ajustan automáticamente la tasa de aprendizaje para cada parámetro del modelo, basándose en la historia de los gradientes. Esto puede mejorar el rendimiento en problemas complejos y hacer que el modelo sea menos sensible a la configuración inicial de la tasa de aprendizaje.

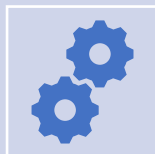
# Optimización de funciones continuas (modelos)

Definición y conceptos básicos

<https://datasmarts.net/es/que-es-un-optimizador-y-para-que-se-usa-en-deep-learning/>

# Optimizadores de primer orden

## Concepto



Los algoritmos de optimización de primer orden implican explícitamente el uso de la primera derivada (gradiente) para elegir la dirección en la que moverse en el espacio de búsqueda.



Los procedimientos implican primero calcular el gradiente de la función y luego seguir el gradiente en la dirección opuesta (por ejemplo, cuesta abajo hasta el mínimo para problemas de minimización) utilizando un tamaño de paso (también llamado tasa de aprendizaje).



Un tamaño de paso demasiado pequeño da como resultado una búsqueda que lleva mucho tiempo y puede atascarse, mientras que un tamaño de paso demasiado grande dará como resultado zigzagueos o rebotes alrededor del espacio de búsqueda, perdiendo completamente el óptimo.

# Optimizadores de primer orden

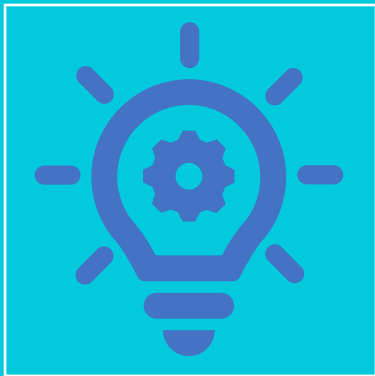
## Concepto

Los algoritmos de primer orden generalmente se denominan descenso de gradiente, y nombres más específicos se refieren a extensiones menores del procedimiento, por ejemplo:

- Gradient Descent
- Momentum
- Adagrad
- Adam

# Optimizadores de modelos

## Concepto



**Cuando se aplican:** Se utilizan durante el entrenamiento de modelos de aprendizaje automático, especialmente en modelos de aprendizaje profundo, para minimizar la función de pérdida, ajustando los pesos del modelo.



**Objetivo:** El objetivo es encontrar el conjunto de parámetros (pesos) del modelo que minimice la función de pérdida, es decir, mejorar el rendimiento del modelo en la tarea específica (como clasificación o regresión).

# GSD

## Concepto



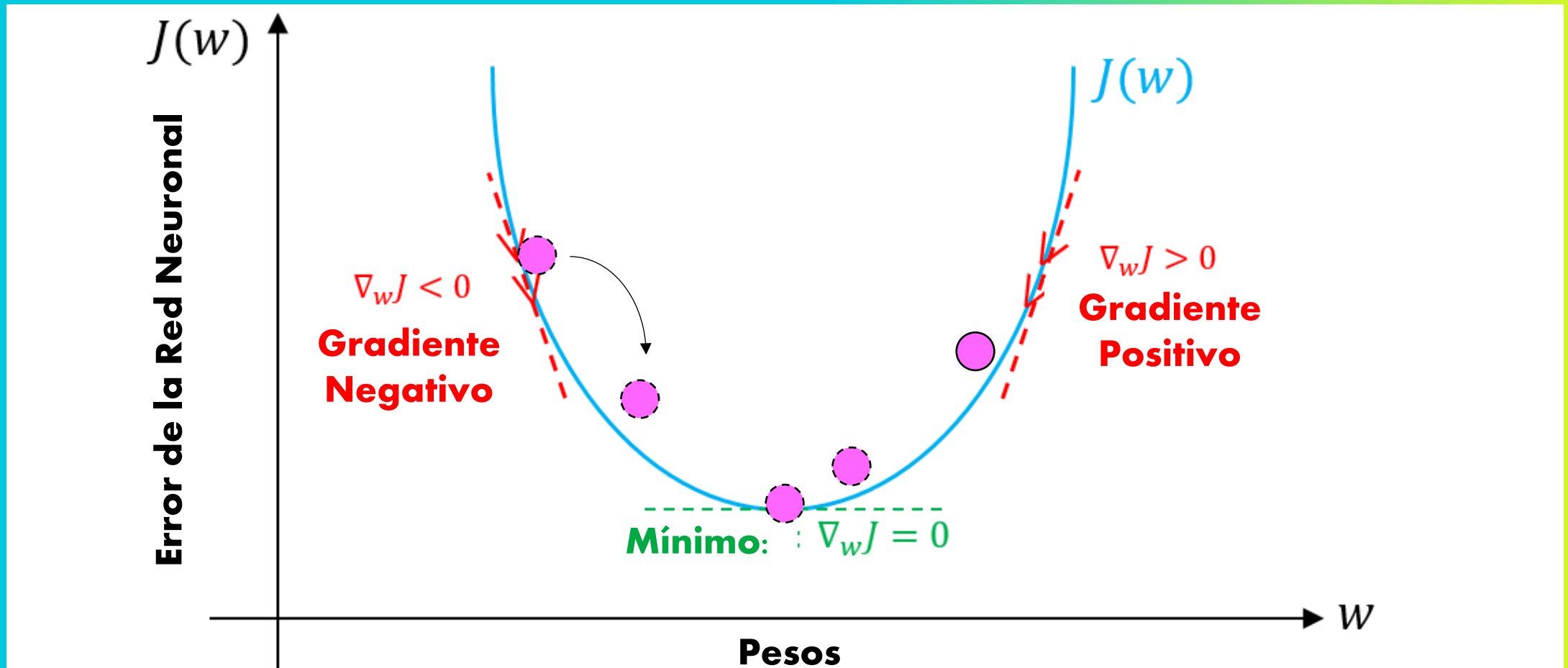
**SGD** es una variante del algoritmo de Gradiente Descendente que, en lugar de utilizar todo el conjunto de datos para calcular el gradiente de la función de pérdida en cada paso (como hace el Gradiente Descendente tradicional), utiliza un solo dato o un subconjunto pequeño (batch) de datos aleatorios.



Esto hace que el cálculo del gradiente y la actualización de los parámetros sean mucho más rápidos y permite el entrenamiento en conjuntos de datos muy grandes.

# Gradiente Descendente

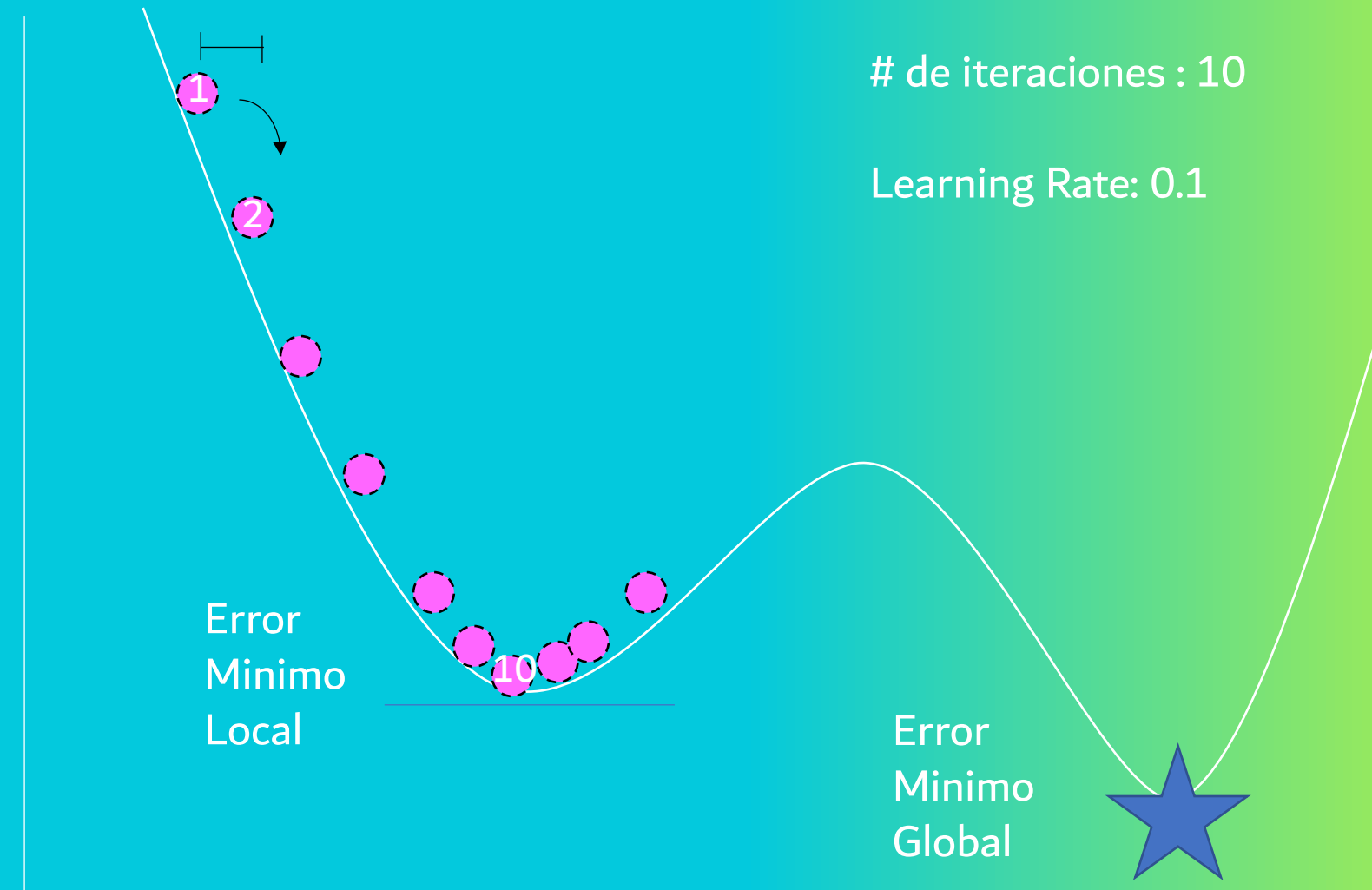
## Deep Learning



Si esta es una función donde **y** fue el error, querríamos encontrar la **x** que lo minimice. Por un momento, imagina que esta no era una función fácil sino una compleja.

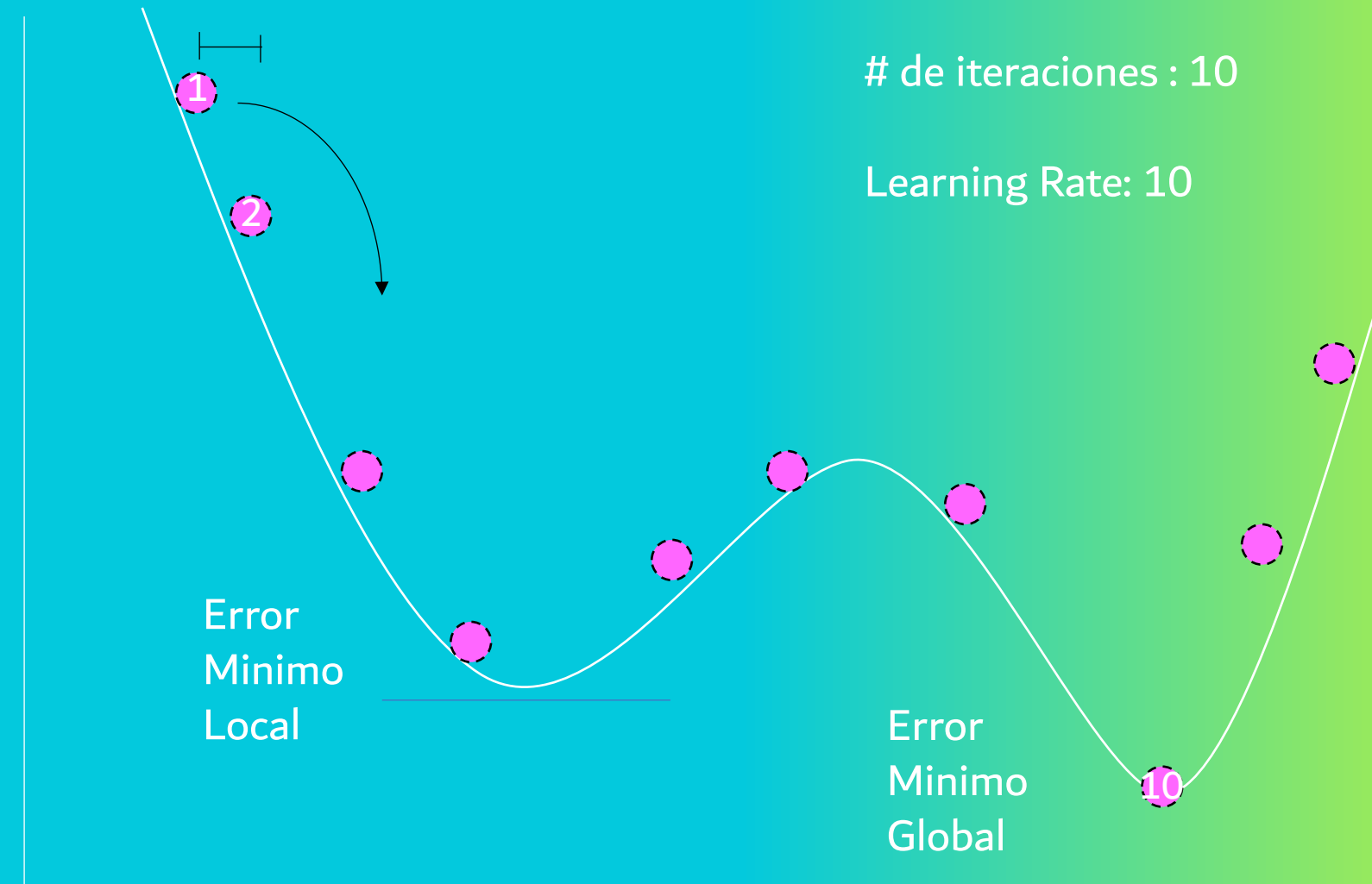
# Gradiente Descendente

## Deep Learning



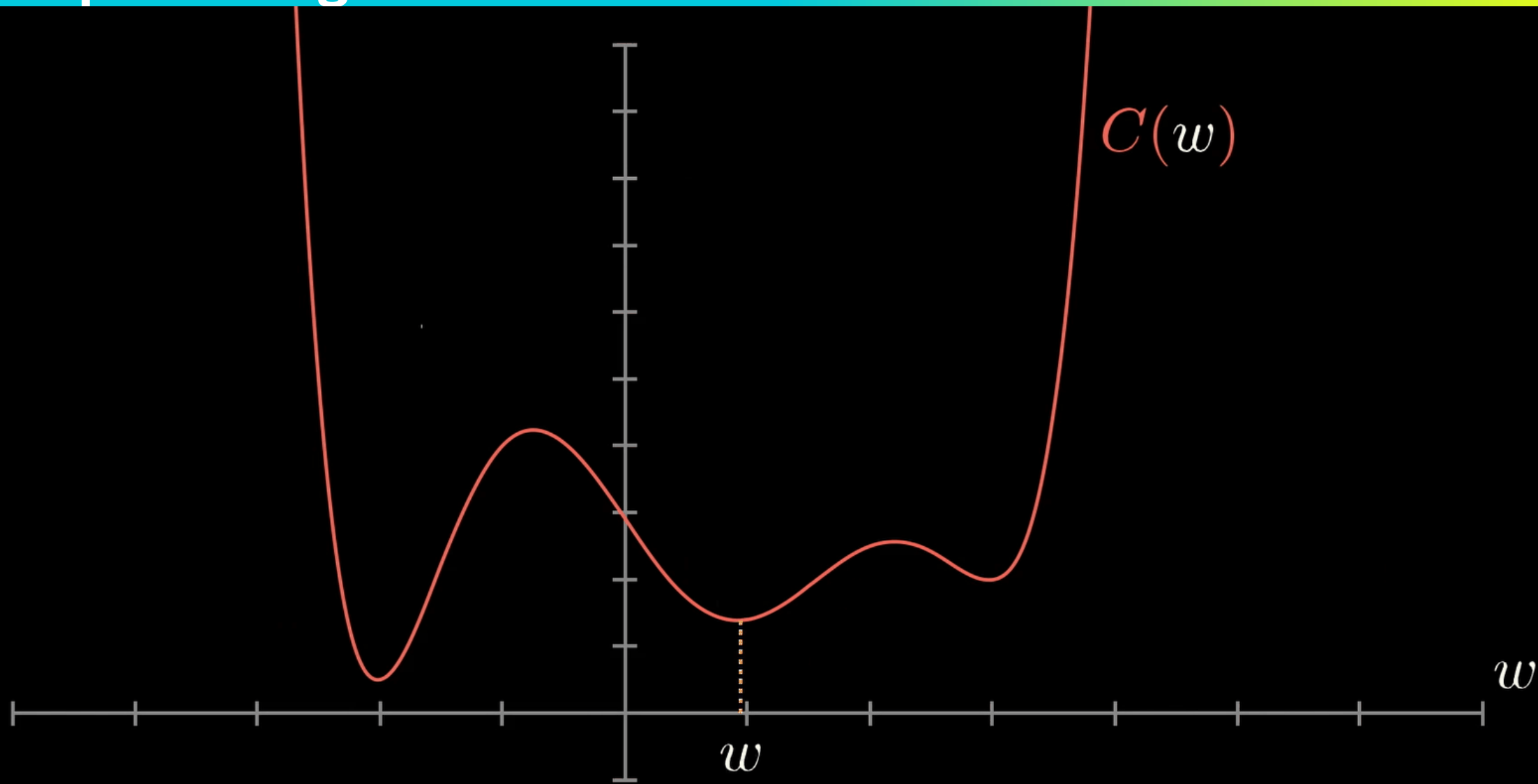
# Gradiente Descendente

## Deep Learning



# Gradiente Descendente

## Deep Learning



# GSD

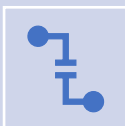
## Lo más importante



**Eficiencia en grandes conjuntos de datos:** Al utilizar solo una muestra del conjunto de datos en cada iteración, SGD puede manejar eficientemente grandes volúmenes de datos.



**Convergencia rápida:** A menudo converge más rápidamente que el Gradiente Descendente en la práctica, especialmente en grandes conjuntos de datos.



**Variabilidad en la convergencia:** Aunque la naturaleza estocástica de SGD introduce ruido en el proceso de optimización (puede "rebotar" alrededor del mínimo de la función de pérdida), esto puede ayudar a escapar de mínimos locales en funciones de pérdida no convexas.

# GSD

## Cuando es mejor aplicarlo



En el entrenamiento de modelos de aprendizaje profundo con grandes conjuntos de datos.



Cuando se necesita un algoritmo de optimización eficiente en memoria y rápido en términos de tiempo por iteración.



En problemas donde está bien tener una cierta cantidad de ruido en el proceso de optimización, lo cual puede ayudar a evitar óptimos locales.

# GSD

## Librerías

**TensorFlow:** a través de `tf.keras.optimizers.SGD`.

**PyTorch:** mediante `torch.optim.SGD`.

**Scikit-learn:** para modelos más tradicionales, usando `sklearn.linear_model.SGDClassifier` o `SGDRegressor`

# GSD

## Datos clave



La tasa de aprendizaje (**learning\_rate**) es un hiperparámetro crucial en SGD. Una tasa muy alta puede hacer que el modelo "salte" sobre el mínimo, mientras que una tasa muy baja puede hacer que el entrenamiento sea muy lento o que el modelo se quede atascado en mínimos locales.

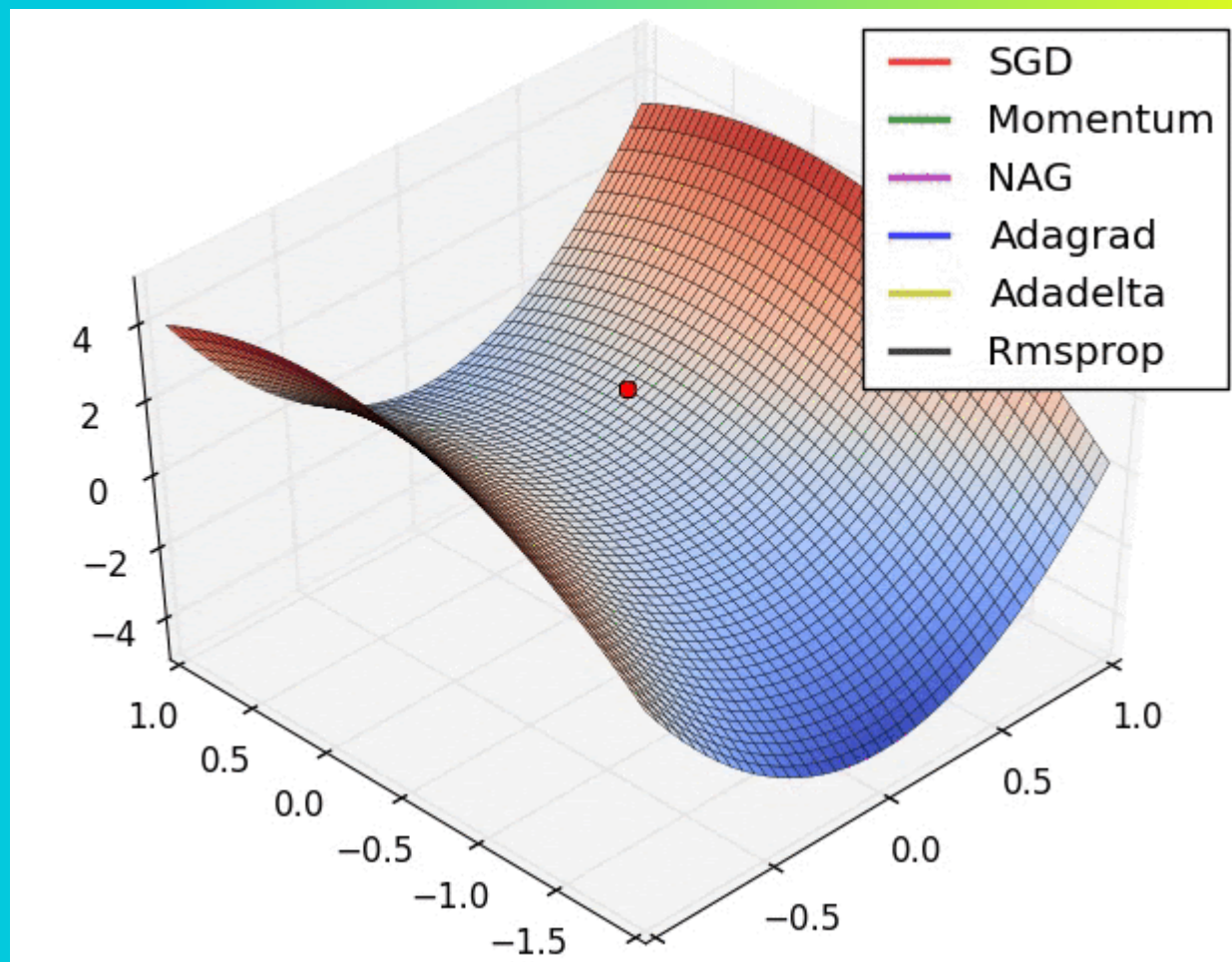


A menudo se utiliza en combinación con técnicas como el momentum para ayudar a acelerar la convergencia y mitigar el problema del ruido en las actualizaciones de los parámetros.

# Adam

## Lo más importante

El **optimizador Adam** (Adaptive Moment Estimation) es un método de optimización ampliamente utilizado en el entrenamiento de modelos de aprendizaje automático, especialmente para redes neuronales profundas. Combina las ventajas de dos otros algoritmos de optimización: RMSprop (Root Mean Square Propagation) y Gradiente Descendente con Momento.



# Adam

## ¿Como funciona?

Adam mantiene dos estimaciones móviles para cada parámetro del modelo:

Los primeros momentos (el promedio) de los gradientes (similares al momento en Gradiente Descendente con Momento).

Los segundos momentos (la varianza no centrada) de los gradientes (similar a RMSprop).

Estas estimaciones se utilizan para adaptar la tasa de aprendizaje de cada parámetro de manera individual, ajustándola según la estimación del primer y segundo momento del gradiente. Esto hace que Adam sea muy efectivo en práctica, especialmente en situaciones con gradientes dispersos o ruidosos, así como para problemas con objetivos muy sensibles a los parámetros o datos de entrada

# Adam

## Lo más importante



**Adaptabilidad:** Adam ajusta la tasa de aprendizaje de cada parámetro individualmente, basándose en estimaciones de los primeros y segundos momentos de los gradientes.



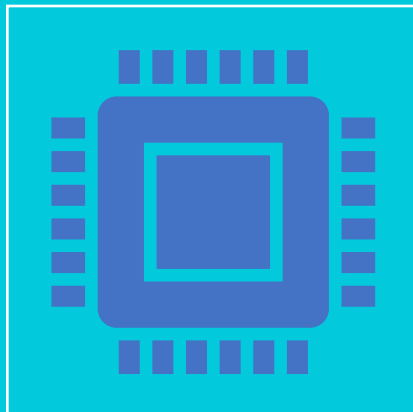
**Eficiencia en problemas complejos:** Muestra un buen desempeño en práctica, especialmente en problemas con altas dimensionalidades y/o donde las funciones de costo son particularmente complejas.



**Requiere poca memoria:** A pesar de mantener estimaciones para los gradientes, el requerimiento de memoria es relativamente bajo

# Adam

## Cuando aplicarlo



En el entrenamiento de redes neuronales profundas, especialmente cuando se dispone de datos complejos y modelos con una gran cantidad de parámetros.



En problemas donde otros optimizadores como SGD no convergen tan rápidamente o quedan atrapados en mínimos locales.

# Adam

## Librerías



**TensorFlow:** a través de `tf.keras.optimizers.Adam`.



**PyTorch:** mediante `torch.optim.Adam`.



**Keras:** como parte de `keras.optimizers.Adam` cuando se usa Keras como API independiente

# Adam

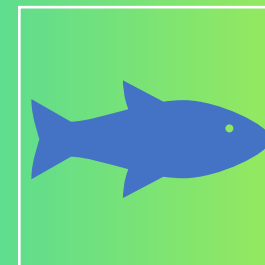
## Datos clave



La tasa de aprendizaje inicial (**learning\_rate**) es un hiperparámetro importante para Adam y suele requerir algo de ajuste según el problema específico.



Adam automáticamente realiza una corrección de sesgo en las estimaciones de los momentos, lo que ayuda a obtener estimaciones más precisas al inicio del entrenamiento y después de los cambios en el gradiente.



Aunque Adam es muy versátil y eficaz, puede que no sea la mejor opción en todos los casos. Experimentar con diferentes optimizadores y configuraciones sigue siendo una buena práctica en el desarrollo de modelos de aprendizaje automático

# Optimizadores

## Resumen

| OPTIMIZADOR      | VENTAJAS                                          | APLICACIONES Y USOS                                                               | PRINCIPALES CARACTERÍSTICAS                                                                            |
|------------------|---------------------------------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| SGD              | - Simple y fácil de implementar.                  | - Bueno para grandes conjuntos de datos y modelos simples.                        | - Actualiza los parámetros utilizando solo un subconjunto de datos a la vez.                           |
|                  | - Bajo consumo de memoria.                        | - Puede manejar problemas de optimización convexos y no convexos.                 | - Requiere una tasa de aprendizaje cuidadosamente ajustada.                                            |
| Gradient Descent | - Convergencia garantizada en problemas convexos. | - Adecuado para conjuntos de datos pequeños y medianos.                           | - Utiliza todo el conjunto de datos para calcular el gradiente en cada paso.                           |
|                  | - Fácil de entender y aplicar.                    | - Útil en problemas con superficies de error suaves y bien comportadas.           | - Puede ser lento para conjuntos de datos grandes debido al cálculo del gradiente en todo el conjunto. |
| Momentum         | - Acelera la convergencia.                        | - Efectivo en superficies de error con curvaturas y mínimos locales.              | - Agrega un término de momento para acelerar los gradientes en la dirección correcta.                  |
|                  | - Reduce la oscilación.                           | - Utilizado en problemas de optimización más complejos y en aprendizaje profundo. | - Puede escapar de mínimos locales más fácilmente que el Gradient Descent puro.                        |

# Optimizadores

## Resumen

| OPTIMIZADOR | VENTAJAS                                                                                                                              | APLICACIONES Y USOS                                                                                                                                       | PRINCIPALES CARACTERÍSTICAS                                                                                                                                                      |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Adagrad     | <ul style="list-style-type: none"><li>- Ajusta la tasa de aprendizaje automáticamente.</li></ul>                                      | <ul style="list-style-type: none"><li>- Útil para tratar con datos dispersos.</li></ul>                                                                   | <ul style="list-style-type: none"><li>- Adapta la tasa de aprendizaje de cada parámetro basándose en los gradientes pasados para ese parámetro.</li></ul>                        |
|             | <ul style="list-style-type: none"><li>- Bueno para problemas con gradientes irregulares.</li></ul>                                    | <ul style="list-style-type: none"><li>- Recomendado para problemas donde algunos parámetros o datos son más informativos que otros.</li></ul>             | <ul style="list-style-type: none"><li>- Reduce la tasa de aprendizaje de manera agresiva, lo que puede llevar a una convergencia prematura.</li></ul>                            |
| Adam        | <ul style="list-style-type: none"><li>- Convergencia rápida.</li></ul>                                                                | <ul style="list-style-type: none"><li>- Aplicado ampliamente en entrenamiento de redes neuronales profundas.</li></ul>                                    | <ul style="list-style-type: none"><li>- Combina los beneficios de Adagrad y RMSprop, ajustando las tasas de aprendizaje basadas en momentos de primer y segundo orden.</li></ul> |
|             | <ul style="list-style-type: none"><li>- Ajustes automáticos de tasa de aprendizaje.</li></ul>                                         | <ul style="list-style-type: none"><li>- Versátil para una variedad de problemas no convexos.</li></ul>                                                    | <ul style="list-style-type: none"><li>- Realiza correcciones de sesgo en los momentos para mejorar la precisión de las estimaciones en las fases iniciales.</li></ul>            |
| RMSprop     | <ul style="list-style-type: none"><li>- Resuelve el problema de la rápida disminución de la tasa de aprendizaje en Adagrad.</li></ul> | <ul style="list-style-type: none"><li>- Útil en entrenamientos de redes neuronales y problemas no convexos.</li></ul>                                     | <ul style="list-style-type: none"><li>- Adapta las tasas de aprendizaje de cada parámetro utilizando la media móvil del cuadrado de los gradientes.</li></ul>                    |
|             | <ul style="list-style-type: none"><li>- Ajusta la tasa de aprendizaje automáticamente.</li></ul>                                      | <ul style="list-style-type: none"><li>- Eficaz en problemas donde es deseable mantener un cierto nivel de exploración durante el entrenamiento.</li></ul> | <ul style="list-style-type: none"><li>- Similar a Adagrad pero con mejoras para evitar una disminución demasiado agresiva de la tasa de aprendizaje.</li></ul>                   |

# Tasa de aprendizaje vs Optimizadores

## Datos clave

| Optimizador | Tasa de Aprendizaje                                        | Ajuste Automático | Sensibilidad a la Tasa Inicial | Características Principales                                                                                                                                                                                   |
|-------------|------------------------------------------------------------|-------------------|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SGD         | Fija o adaptable (necesita ajuste manual o decaimiento)    | No                | Alta                           | - Simple y transparente.<br>- Requiere ajuste manual de la tasa.<br>- Puede necesitar decaimiento de la tasa de aprendizaje.                                                                                  |
| Adam        | Ajuste automático basado en los momentos de los gradientes | Sí                | Baja                           | - Ajusta la tasa de aprendizaje por parámetro basado en estimaciones del primer y segundo momento.<br>- Menos sensible a la tasa inicial.<br>- Buen rendimiento general sin mucha necesidad de ajuste manual. |
| Adagrad     | Ajuste automático individual para cada parámetro           | Sí                | Baja                           | - Adecuado para datos dispersos y parámetros con diferentes frecuencias de actualización.<br>- La tasa de aprendizaje efectiva disminuye continuamente, lo que puede llevar a una convergencia prematura.     |